Together First, Apart Later? When Optimizers Start to Learn Different Functions

Project report, MIT 6.7910 Statistical Learning Theory, Fall 2026

Juntang Wang Víctor Conchello

Draft, October 30, 2026

Abstract

From the same initialization and the same sequence of mini-batches, we train small transformer language models with SGD with momentum, AdamW and Muon, and ask when their learned functions start to differ. We measure the distance between the next-token predictions of two runs on held-out text, compare runs at equal training loss so that speed is not mistaken for difference, and calibrate against the distance between two runs of the same optimizer that differ only in data order. For each pair of optimizers we look for a persistent split, a loss level from which their distance stays above this floor, with a bootstrap over data orders, and test what coincides with the changes in distance: sharpness reaching its stability threshold, the end of warmup, or a loss level. In all three language-model settings and for every pair, the loss-matched distance is 30 to 200 times the floor at high loss and falls back to near the floor around the stage where the models pass through the bigram statistics of the data. On TinyStories it stays there; on Shakespeare AdamW and Muon have one more short episode, AdamW–SGD stays at 1.3 to 1.7 times the floor, and Muon–SGD, where SGD barely trains, has the only persistent split, which is lost with another metric or a stricter threshold. At equal steps the optimizers do differ, mostly because they progress at different speeds; at the end of TinyStories-30M, replacing AdamW by Muon changes the predictions no more than changing the data order. On TinyStories AdamW sits at its adaptive edge of stability for much of training whether or not the optimizers differ, and SGD reaches its threshold only early in training. On CIFAR-10 we replicate the break-even effects of Jastrzebski et al. (2020) on sharpness and gradient noise, but at equal loss, away from the initial loss, two learning rates differ by at most about twice the floor. When we switch optimizer at nine times during training, the run ends at the function of the new optimizer whenever that optimizer is fast enough to get there in the remaining steps, which on TinyStories is always the case when it is the faster of the two, so the endpoint is not decided early. Weight space keeps a mark that function space does not: after a change of optimizer a fork needs a later fork step to stay linearly connected to its parent than after a change of data order (except for Muon on Shakespeare), the loss barrier between AdamW and Muon extends beyond the first block, where that between two data orders lies, and the weights carry each optimizer's geometry, mixed after a swap according to how far each optimizer moved them. On TinyStories the hidden representations of AdamW and Muon differ after the first block and little after the others, and their functions differ no more than with another data order even off the data distribution. A second initialization reproduces the results, the other learning rates of the sweeps give the same comparison on TinyStories, and on a linear model with known, different limits the same test does find a split, near interpolation; AdamW and Muon trained into memorization on Shakespeare still do not split.

1 Introduction

The optimizer decides which of the many solutions with equal training loss a network converges to. In linear models this choice is understood: a first-order method acts as steepest descent in some norm and converges in direction to the max-margin solution in that norm (Gunasekar et al., 2018), which is ℓ_2 for gradient descent (Soudry et al., 2018), ℓ_∞ for Adam (Zhang et al., 2024a; Xie and Li, 2024) and the spectral norm for Muon (Fan et al., 2025). These are statements about the limit of infinite training, approached slowly. In deep networks the comparison has been made at the end of training: adaptive methods were found to generalize worse (Wilson et al., 2017), then to learn richer features (Vasudeva et al., 2025); models with the same error can compute different functions (Cortesi et al., 2026); and on language models most optimizers reach a similar loss (Zhao et al., 2025).

We ask *when* the difference between optimizers appears. From the same initialization and the same data order, do SGD with momentum, AdamW and Muon first follow the same path in function space and only later separate? If so, where is the split, what happens in the optimization at that moment, and is the final difference already decided there?

The timing matters for two reasons. First, it says which theory is relevant. If optimizers separate early, the asymptotic margin results do not describe how the difference arises, and early events in training are candidates instead. In vision, Jastrzebski et al. (2020) show that SGD runs with different learning rates separate in prediction space early in training,

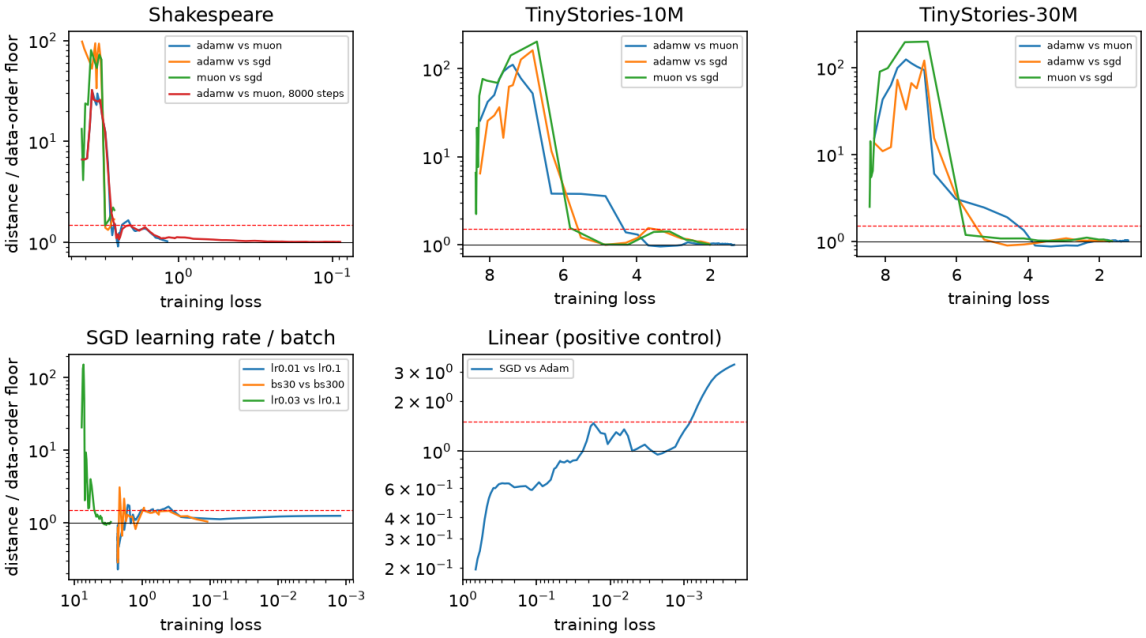


Figure 1: Overview. Function distance between two optimizers (or two SGD learning rates or batch sizes) at equal training loss, over the distance between two runs of one of them with different data orders (training progresses to the right). The red line is the threshold $1 + m_\times = 1.5$ of the split rule. In all language-model settings the ratio is largest at the start and falls to about 1 by the bigram stage; the one persistent split, Muon–SGD on Shakespeare, is where SGD stalls. The same holds for SGD learning rates and batch sizes on CIFAR-10 and TinyStories-10M (bottom left). Red, top left: AdamW–Muon trained to memorization (8000 steps†). Bottom middle: a linear model on separable data, where gradient descent and Adam have different limits†; there the rule finds a persistent split, near interpolation.

and tie this to a *break-even point*; and Frankle et al. (2020) find that training becomes stable to the noise of SGD early, after which copies of a network trained with different data orders stay in one linearly connected region. Edge-of-stability events (Cohen et al., 2021; Cohen et al., 2022; Andreyev and Beneventano, 2024) are the natural candidate for such an early event, and their thresholds differ by optimizer. Second, it matters in practice. Switching optimizers during training (Keskar and Socher, 2017) assumes the second optimizer can still change the solution; a split point bounds when that is possible.

Approach. We train nanoGPT models (Karpathy, 2022) on character-level Shakespeare (10.8M parameters) and on TinyStories (14M and 30M parameters), with SGD with momentum, AdamW, and Muon (Jordan et al., 2024), each at a learning rate chosen by a sweep, from a shared initialization, with five data orders per optimizer. The distance between two runs is the Jensen–Shannon divergence and the top-1 disagreement of their next-token predictions on fixed held-out contexts. We compare runs at equal steps and at equal training loss, and calibrate against a noise floor: the distance between two runs of the same optimizer with different data orders. A SimpleCNN on CIFAR-10 replicates the break-even analysis of Jastrzebski et al. (2020) as a check of the pipeline.

Contributions.

1. **Divergence curves** between every pair of optimizers in three settings, at equal steps and at equal training loss, against a data-order noise floor (Section 5.2). In all 9 pairs the loss-matched distance is far above the floor early and returns toward it by the bigram stage of training; it stays near it on TinyStories, and on Shakespeare apart from a short AdamW–Muon episode and the pairs with SGD.
2. **A test for a split point**: episodes† in which the distance exceeds the floor and a persistent split†, located as a loss level with a bootstrap over data orders (Sections 4.2 and 5.2). There is a persistent split in 1 of 9 pairs (Muon–SGD on Shakespeare, from loss 2.82, in 96.5% of resamples), none between AdamW and Muon.
3. **Optimizer against data order**† at equal steps (Section 5.3): at the end of TinyStories-30M the AdamW–Muon distance equals the data-order floor, and the n-gram references show where the optimizers' routes meet.
4. **What coincides with the changes in distance**: sharpness against its optimizer-specific stability threshold, batch sharpness, the end of warmup (Section 5.4). The edge of stability does not explain them: on TinyStories AdamW is at its threshold for long stretches whether or not the optimizers differ, and SGD only early in training. Muon with its auxiliary AdamW matched to AdamW† still memorizes on Shakespeare, so the memorization comes

from its updates of the hidden matrices.

5. **A causal test by swapping** optimizers at nine fork times with warm optimizer states, scored by where the fork lands between the two pure runs, and a basin-level test by linear mode connectivity (Section 5.5). On TinyStories-10M the optimizer that trains last decides the function whenever it is fast enough to: forks from SGD to AdamW or Muon land on the new optimizer even after 4000 steps, while forks into SGD end closer to their parent than to pure SGD from step 1500 on (step 5000 on TinyStories-30M, where SGD's rate is higher). In weight space the optimizer leaves a mark that function space does not show: forks that change the optimizer stay linearly connected to their parent only from a later fork step than forks that change the data order (except for Muon on Shakespeare), and between pure runs† the barrier between AdamW and Muon extends beyond the first block, where that between two data orders lies.
6. **The anatomy of the sharp direction**: which parameter blocks carry the top Hessian eigenvector and its curvature, by optimizer and over training (Section 5.6). It differs by optimizer (embeddings for Muon on Shakespeare, value–output matrices for Muon on TinyStories, head or gains for AdamW on TinyStories), and most of the eigenvalue usually comes from curvature between blocks.
7. **A replication of the break-even point** on CIFAR-10 and its transfer to the language models (Section 5.7). The sharpness and gradient-noise effects replicate, but at equal training loss, away from the initial loss, two learning rates differ by at most about twice the floor.
8. **Controls**† (Section 5.9): a second initialization moves the function as much as a new data order and reproduces the episodes; SGD's loss gap on rare tokens is a matter of speed; and on a linear model, where theory predicts different limits, the same detector finds a persistent split near interpolation, while AdamW and Muon trained into memorization on Shakespeare still do not split.

Planned and added analyses. The project plan comprised the learning-rate sweeps, the divergence curves at equal steps and at equal loss against the data-order floor, the first exceedance t^* with a bootstrap over data orders, its comparison with the sharpness thresholds and the end of warmup, the swaps with warm optimizer states scored by distance and function metrics, linear mode connectivity against the fork time, the anatomy of the top eigenvector, and the CIFAR-10 break-even check. Everything else was added after the first results, mostly to test explanations of them, and is marked with † in the headings and captions: the episodes and the persistent split (as t^* turned out to be degenerate), the decomposition into optimizer and data order and the n-gram references, the edge of stability of Muon's auxiliary AdamW and the matched control, the robustness checks, the anatomy of AdamW's preconditioned operator, the floor-normalized swap scores and the barriers to pure runs, the scale-free weight metrics ($\|\theta\|_\infty/\|\theta\|_2$ and the stable rank) and their shifts under swaps, the representation similarity, the layer-wise barriers, the comparison off the data distribution, the break-even runs on the language models and with two batch sizes, and the controls of Section 5.9 (a second initialization, token frequency, a linear positive control, Shakespeare runs trained into memorization, and other learning rates).

Code: `https://github.com/qqgjyx/MIT-6.7910` (`code/`).

2 Related work

Implicit bias of optimizers. For linear models the limit of training depends on the geometry of the optimizer: steepest descent in a norm converges to the max-margin solution in that norm (Gunasekar et al., 2018), giving ℓ_2 for gradient descent (Soudry et al., 2018), ℓ_∞ for Adam (Zhang et al., 2024a), an ℓ_∞-constrained KKT point for AdamW with weight decay (Xie and Li, 2024), and the spectral norm for Muon (Fan et al., 2025). Away from the limit, finite step sizes act as implicit regularizers: for SGD the modified loss penalizes the gradient norm (Smith et al., 2021), while for Adam the corresponding term can have the opposite sign (Cattaneo et al., 2024). Empirical comparisons in deep networks are made at the end of training (Wilson et al., 2017; Vasudeva et al., 2025; Zhao et al., 2025; Cortesi et al., 2026). We look at the whole trajectory instead.

Edge of stability. With full-batch gradient descent, the top Hessian eigenvalue rises until it reaches $2/\eta$ and then stays near it while the loss keeps decreasing (Cohen et al., 2021); this is explained by self-stabilization (Damian et al., 2023) and described by central flows (Cohen et al., 2025). Adaptive methods reach an analogous *adaptive edge of stability*, where the top eigenvalue of the preconditioned Hessian sits at $(2 + 2\beta_1)/((1 - \beta_1)\eta)$ (Cohen et al., 2022). For mini-batch SGD, Andreyev and Beneventano (2024) show that the quantity that stabilizes at $2/\eta$ is the *batch sharpness*, the curvature of mini-batch losses along their own gradients, while $\lambda_{\max}$ stays below; Andreyev et al. (2026) study the same regime with momentum. Kalra and Barkeshli (2024) relate learning-rate warmup to these sharpness dynamics. Transformer Hessians differ strongly across parameter blocks (Zhang et al., 2024b), which motivates our block decomposition.

Table 1: Settings. L layers, model width d, vocabulary V. Parameters in parentheses exclude the token and position embeddings and the head. LM batches are sequences × tokens. "aux.": learning rate of Muon's auxiliary AdamW. CIFAR-10 settings in Section 4.9. LM schedules: linear warmup for w steps, then cosine decay to 10% of the peak learning rate at the last step. The CIFAR-10 runs use a constant learning rate.

Setting	Model (L, heads, d, V)	Params	Steps	Batch	w	η_{SGD}	η_{AdamW}	η_{Muon} (aux.)
Shakespeare-char	6, 6, 384, 65	10.8M (10.6M)	1,500	64×256	100	0.05	$3 \cdot 10^{-4}$	0.04 (10^{-3})
TinyStories-10M	6, 6, 384, 4096	13.9M (10.6M)	10,000	128×256	200	0.03	$2 \cdot 10^{-3}$	0.01 ($2 \cdot 10^{-3}$)
TinyStories-30M	8, 8, 512, 4096	29.5M (25.2M)	20,000	128×256	400	0.1	$2 \cdot 10^{-3}$	0.01 ($1.5 \cdot 10^{-3}$)
CIFAR-10	SimpleCNN, ReLU	0.36M	39,000	128	–	plain SGD, $\eta \in \{0.001, 0.01, 0.1\}$		

Break-even point and early training. Jastrzebski et al. (2020) argue that early in training SGD reaches a break-even point, after which the learning rate and batch size set the curvature and the gradient noise along the rest of the trajectory, and show that SGD runs with different learning rates separate early in prediction space. Frankle et al. (2020) find that training becomes stable to SGD noise early: two copies trained on from the same checkpoint with different data orders end in one linearly connected region once the checkpoint is past an early point. We use both as templates, comparing optimizers instead of learning rates or data orders.

Muon and optimizer switching. Muon (Jordan et al., 2024) updates each hidden weight matrix with its momentum after approximate orthogonalization by a Newton–Schulz iteration, and has been scaled to large language models (Liu et al., 2025). SWATS (Keskar and Socher, 2017) switches from Adam to SGD during training, scored by accuracy; our swaps are scored by where the function ends.

3 Setup

Models and data. Table 1 lists the four settings. The language models are nanoGPT (Karpathy, 2022) with pre-LayerNorm blocks, GELU MLPs of width $4d$, no biases, no dropout, an untied output head, and separate query, key and value projections (so that Muon orthogonalizes each and the block decomposition of Section 4.7 can separate the query-key from the value-output matrices). The context is 256 tokens. Character-level Shakespeare uses nanoGPT's 90/10 split and a 65-character vocabulary. TinyStories (Eldan and Li, 2023) (GPT-4 version) uses a 4096-token byte-level BPE trained on the training split, so that the head and the stored predictions stay small. The vision check is the SimpleCNN of Jastrzebski et al. (2020) (two 32-filter and two 64-filter 3×3 convolutions with max pooling after each pair, then a 128-unit dense layer, ReLU) on CIFAR-10 (Krizhevsky, 2009) without augmentation.

Shakespeare runs take 24.6M tokens (about 24 passes over the 1M training characters). Without dropout the validation loss is lowest near step 1000 and rises after, so these runs stop at 1500 steps. The TinyStories runs take 328M and 655M tokens (about 0.6 and 1.2 epochs).

Optimizers. Write g_t for the mini-batch gradient at step t and $\eta_t = \eta\,\varphi(t)$ for the learning rate, with $\varphi(t) = (t+1)/w$ for $t < w$ and $\varphi(t) = 0.1 + 0.45\,(1 + \cos(\pi\,(t-w)/(T-w)))$ after, T being the last step.

- **SGD** with heavy-ball momentum $\mu = 0.9$ and no weight decay: $b_{t+1} = \mu b_t + g_t$, $\theta_{t+1} = \theta_t - \eta_t b_{t+1}$.
- **AdamW** with $(\beta_1, \beta_2) = (0.9, 0.95)$ ($(0.9, 0.99)$ for Shakespeare, as in nanoGPT), $\epsilon = 10^{-8}$, and decoupled weight decay 0.1 on every tensor with at least two dimensions (including embeddings and head) and none on LayerNorm gains: $\theta_{t+1} = (1 - \eta_t \lambda_{\text{wd}})\theta_t - \eta_t\,\hat{m}_{t+1}/(\sqrt{\hat{v}_{t+1}} + \epsilon)$, with bias-corrected moments $\hat{m}, \hat{v}$.
- **Muon** (Jordan et al., 2024) on every matrix inside the transformer blocks (query, key, value, attention output, both MLP matrices), with momentum $\beta = 0.95$, Nesterov, and no weight decay: $M_{t+1} = \beta M_t + (1-\beta)G_t$, $W_{t+1} = W_t - \eta_t \sqrt{\max(1, n_{\text{out}}/n_{\text{in}})}\,\text{NS}_5\big((1-\beta)G_t + \beta M_{t+1}\big)$, where NS_5 is five steps of the quintic Newton–Schulz iteration that approximates the orthogonal polar factor. Embeddings, head and gains are updated by an auxiliary AdamW with $(0.9, 0.95)$, $\epsilon = 10^{-8}$, no weight decay, and its own learning rate (Table 1).

Gradient clipping is off, since it would hide edge-of-stability dynamics. Training steps of the language models use bfloat16 autocast; all measurements (Section 4) are in float32.

Learning rates. For each setting and optimizer we ran five learning rates (grids in Table 5) on data order 0, with the schedule and length of the main runs and no measurements, and kept the one with the best final validation loss (on the validation sequences of the probe, Section 4.1). Shakespeare is the exception: since its runs overfit, we kept the best *minimum* validation loss over training. SGD is the exception to the rule: in every setting its best rate on data order 0 diverged or blew up on other data orders (0.2 on Shakespeare, 0.1 on TinyStories-10M, 0.3 on TinyStories-30M), so

SGD uses a lower rate: 0.05 on Shakespeare (the smallest of its grid) and the next lower rate on TinyStories, 0.03 and 0.1. On Shakespeare every SGD rate of the grid stops at about the same minimum validation loss (2.44–2.50, the bigram level), so we took the smallest and most stable one. Runs whose training loss became non-finite were stopped and counted as diverged.

Shared initialization and data order. Every run starts from the same initialization (seed 0). The mini-batch at step t depends only on the data order d and on t: for the language models it is B windows of 256 tokens at offsets drawn uniformly from the training split by a generator seeded with (d, t); for CIFAR-10 it is a slice of a per-epoch permutation seeded with (d, epoch), dropping the last incomplete batch. Two runs with the same d therefore see identical batches whatever their optimizer, and forks and resumed runs see the batches their parent would have seen. Batches used for measurements come from separate random streams and never change the training order. Each optimizer is run with data orders $d \in \{0, \dots, 4\}$: 15 main runs per setting. The runs with $d = 0$ also keep the other optimizers' states in shadow and save fork checkpoints (Section 4.5).

Control for Muon's auxiliary Adam.† On Shakespeare, Muon's auxiliary AdamW and the AdamW runs differ in learning rate (10^{-3} against $3 \cdot 10^{-4}$), β_2 (0.95 against 0.99) and weight decay (none against 0.1). A control suite repeats Muon with its auxiliary AdamW matched to AdamW (learning rate $3 \cdot 10^{-4}$, $\beta = (0.9, 0.99)$, weight decay 0.1, which here also applies to the gains), with data orders 0–4 ("matched Muon"). It differs from AdamW only in how the hidden matrices are trained (and in decay on the gains).

4 Methods

Step s always refers to the model θ_s after s updates ($s = 0$ is the initialization). Each run logs predictions at its prediction steps $\mathcal{S}$ and curvature at its measurement steps $\mathcal{M}$; both grids combine a linear and a log-spaced grid (Table 7).

4.1 Function distance and alignment

Distance. Every run stores, at each $s \in \mathcal{S}$, its next-token distributions on a fixed set of N held-out positions, identical across runs: for Shakespeare all 256 positions of 64 validation sequences ($N = 16{,}384$), for TinyStories 16 evenly spaced positions of 128 validation sequences ($N = 2{,}048$), and for CIFAR-10 the 10,000 test images. Write $p_i^X(s)$ for the distribution of run X at step s on position i. The distance between run X at step s and run Y at step s' is the mean Jensen–Shannon divergence in bits,

$$D_{\mathrm{JS}}(X_s, Y_{s'}) = \frac{1}{N} \sum_{i=1}^{N} \mathrm{JS}\big(p_i^X(s),\, p_i^Y(s')\big), \qquad \mathrm{JS}(p, q) = \tfrac{1}{2}\mathrm{KL}(p \,\|\, m) + \tfrac{1}{2}\mathrm{KL}(q \,\|\, m), \quad m = \tfrac{1}{2}(p + q), \tag{1}$$

with base-2 logarithms, so $D_{\mathrm{JS}} \in [0, 1]$. The second distance is the top-1 disagreement $D_{\mathrm{dis}}(X_s, Y_{s'}) = 1 - \frac{1}{N}\sum_i \mathbf{1}\{\arg\max_v p_{i,v}^X(s) = \arg\max_v p_{i,v}^Y(s')\}$.

Alignment by step. $D(X_s, Y_s)$ at the steps $s \in \mathcal{S}$ that both runs log.

Alignment by training loss. Optimizers progress at different speeds, and comparing at equal steps confounds "faster" with "different". Each run therefore also logs, at each $s \in \mathcal{S}$, its mean cross-entropy $\ell_X(s)$ on a fixed training probe, the same for all runs (64 training sequences, all positions, for the language models; 2,000 training images for CIFAR-10). We make it monotone with a running minimum, $\bar{\ell}_X(s) = \min\{\ell_X(s') : s' \in \mathcal{S},\ s' \le s\}$. Each step s of X is matched with the earliest step of Y whose loss is closest,

$$\pi_{XY}(s) = \min\Big(\arg\min_{s' \in \mathcal{S}} \big|\bar{\ell}_Y(s') - \bar{\ell}_X(s)\big|\Big), \qquad \text{kept if } \big|\bar{\ell}_Y(\pi_{XY}(s)) - \bar{\ell}_X(s)\big| \le \tau\, \bar{\ell}_X(s), \tag{2}$$

with $\tau = 0.02$. The kept matches give points $\big(\bar{\ell}_X(s),\, D(X_s, Y_{\pi_{XY}(s)})\big)$, and the *loss-matched distance* $D^{\mathrm{loss}}_{X,Y}(\ell)$ is their piecewise-linear interpolation in ℓ (where several steps share a loss value, the earliest is used), undefined outside the range of kept losses. The loss coordinate is that of the first run, X.

Noise floor. Two runs of the same optimizer from the same initialization with different data orders, $D^{\mathrm{loss}}_{O_d, O_{d'}}$ with $d \neq d'$, differ only through the sampling of mini-batches. A difference between optimizers counts only where it exceeds this floor.

4.2 Locating the split

Fix two optimizers A and B and write A_d, B_d for their runs with data order $d \in \mathcal{D} = \{0, \dots, 4\}$. Runs with the same d share initialization and batches, so they form the *cross* pairs; runs of one optimizer with different d form the *floor* pairs:

$$c_d(\ell) = D^{\text{loss}}_{A_d,B_d}(\ell), \qquad f^O_{dd'}(\ell) = D^{\text{loss}}_{O_d,O_{d'}}(\ell) \quad (O \in \{A, B\},\ d < d').$$

Loss grid. Let $\ell_1 > \ell_2 > \cdots > \ell_J$ be the distinct values of $\bar{\ell}_{A_0}(s)$, $s \in \mathcal{S}$, that lie in the range covered by all $2|\mathcal{D}|$ runs,

$$\Big[\max_X \min_{s\in\mathcal{S}} \bar{\ell}_X(s),\ \min_X \max_{s\in\mathcal{S}} \bar{\ell}_X(s)\Big].$$

Decreasing loss is the order of training.

Statistic. For a multiset S of data orders with distinct elements $U(S)$, let

$$\bar{c}_S(\ell) = \text{mean}_{d\in S}\ c_d(\ell), \qquad F_S(\ell) = \max\Big(\text{mean}_{d<d'\in U(S)}\, f^A_{dd'}(\ell),\ \text{mean}_{d<d'\in U(S)}\, f^B_{dd'}(\ell)\Big), \tag{3}$$

where the cross mean counts repeated orders with their multiplicity, means skip undefined values, and the maximum takes the defined side if only one is. The floor is thus the larger of the two optimizers' average seed-to-seed distances. A level is *above the floor* if the cross distance exceeds the floor by a margin,

$$o_S(j) = \mathbf{1}\big\{\bar{c}_S(\ell_j) > (1 + m_\times)\, F_S(\ell_j) + m_+\big\}, \tag{4}$$

with $m_\times = 0.5$ and $m_+ = 0$; a comparison with an undefined value counts as false.

First exceedance. The first grid level from which the cross distance is above the floor at $k = 3$ consecutive levels,

$$t^*(S) = \ell_{j^*}, \qquad j^* = \min\big\{j :\ o_S(i) = 1 \ \text{ for } i = j, \dots, j + k - 1\big\}, \tag{5}$$

undefined if no such j exists. Since the grid is made of the losses of A_0 at its prediction steps, the k levels are consecutive distinct loss values of that run. This was the planned definition of the split, but it is degenerate: at initialization all runs compute the same function, so the floor is close to 0 for the first steps, while two different optimizers move apart from their first update. In every pair of optimizers t^* is one of the first grid levels, reached at steps 1–8 (Table 2). It records that two optimizers differ from the start, not that they split. We therefore also report:

Episodes and persistent split.† An *episode* is a maximal run of consecutive levels $\ell_{j_0}, \dots, \ell_{j_1}$ with $o_S(j) = 1$ that contains at least k levels ($j_1 - j_0 + 1 \geq k$), reported as the loss range $[\ell_{j_0}, \ell_{j_1}]$. The *persistent split* $t^*_{\text{p}}(S)$ is the start ℓ_{j_0} of the last episode if that episode ends at the last level of the grid ($j_1 = J$), and undefined otherwise: the loss from which the cross distance stays above the floor down to the lowest loss that all runs reach. A persistent split is what "together first, apart later" predicts; an episode that ends before ℓ_J is a difference that later disappears. Figures also show the ratio $\bar{c}_{\mathcal{D}}(\ell)/F_{\mathcal{D}}(\ell)$ against the loss, with the threshold $1 + m_\times$.

Estimate and uncertainty. The point estimates are $t^* = t^*(\mathcal{D})$, $t^*_{\text{p}} = t^*_{\text{p}}(\mathcal{D})$ and the episodes of $\mathcal{D}$. For their uncertainty we draw 1000 bootstrap resamples S_b of $|\mathcal{D}|$ data orders with replacement and report, for t^* and t^*_{p} separately, the 2.5 and 97.5 percentiles of the defined values, together with the fraction of resamples in which one is found (a resample with a single distinct order has no floor and gives none). The distance is D_{JS}; D_{dis} and other values of $m_\times$, k and τ are checked in Section 5.2 (300 resamples each).

In steps. t^* is a loss level, which makes it comparable across optimizers that reach it at different steps. For each run X, $t^*_X = \min\{s \in \mathcal{S} : \bar{\ell}_X(s) \leq t^*\}$ is the first step at which X reaches it, and likewise $t^*_{\text{p},X}$ for the persistent split.

4.3 Optimizer against data order, and n-gram references†

Decomposition at equal steps. For two optimizers X, Y with data orders $d = 0, \dots, n-1$ ($n = 5$) and $d^+ = (d+1) \bmod n$, we compare at each shared prediction step s four families of n pairs: $D(X^d_s, Y^d_s)$ (other optimizer, same data: the cross pairs above), $D(X^d_s, Y^{d^+}_s)$ (other optimizer, other data), and the floors $D(X^d_s, X^{d^+}_s)$ and $D(Y^d_s, Y^{d^+}_s)$ (same optimizer, other data), and plot the median and range over d. If the same-data distance is no larger than the floors, the change of optimizer changes the function no more than a change of data order; if it is smaller than the other-data distance, sharing the batches keeps the two optimizers closer.

N-gram references. From the counts $n(b)$ and $n(a,b)$ of tokens and of consecutive token pairs in the training split we build a unigram model $p_1(b) \propto n(b)+1$ and a bigram model smoothed towards it, $p_2(b \mid a) = \big(n(a,b)+\alpha\, p_1(b)\big)/\big(n(a)+\alpha\big)$ with $\alpha = 1$, evaluated at the validation probe positions (the bigram conditioned on the current token). For a run X we plot D_{JS} (1) between $p^X(s)$ and each reference against the training loss $\bar{\ell}_X(s)$. A minimum near zero means the run passes through that n-gram solution; two optimizers that pass through it at the same loss meet there in function space.

4.4 Sharpness and stability thresholds

At each $s \in \mathcal{M}$, before the update of that step, we measure on θ_s (in float32):

Sharpness. $\lambda_{\max}$ is the largest eigenvalue of the Hessian $H = \nabla^2 L_{\mathcal{H}}(\theta_s)$ of the mean cross-entropy on a fixed batch $\mathcal{H}$ of training examples (32 sequences of 256 tokens for the language models; 2,500 images, 5% of the training set, for CIFAR-10), computed by Lanczos with full reorthogonalization (20 iterations from a fixed random start) on Hessian-vector products.

SGD threshold. On a quadratic with curvature λ, heavy-ball momentum as above is stable if and only if $\eta\lambda < 2+2\mu$. We compare $\lambda_{\max}(\theta_s)$ with $(2+2\mu)/\eta_s$, which is $3.8/\eta_s$ for $\mu = 0.9$ and $2/\eta$ for the plain SGD of the break-even runs.

AdamW preconditioned sharpness. Following Cohen et al. (2022), let $P = (1-\beta_1^t)\,\mathrm{diag}(\sqrt{\hat{v}}+\epsilon)$, with $\hat{v} = v/(1-\beta_2^t)$ built from the optimizer's current second-moment state and step count t. The AdamW update is then $-\eta_t P^{-1} m$ with m the uncorrected first moment, so with P frozen and weight decay ignored it is heavy-ball momentum with coefficient β_1 and step $\eta_t(1-\beta_1)$ in the metric P. It is stable if $\lambda_{\max}(P^{-1}H) < (2+2\beta_1)/((1-\beta_1)\eta_t)$, which is $38/\eta_t$ for $\beta_1 = 0.9$. We compute the preconditioned sharpness $\lambda_{\max}(P^{-1/2}HP^{-1/2})$, which has the spectrum of $P^{-1}H$, by the same Lanczos procedure, for every run that trains with AdamW (including forks into AdamW).

Batch sharpness. Following Andreyev and Beneventano (2024, Def. 1),

$$\mathrm{BS}(\theta) = \frac{\sum_{j=1}^{n} g_j^\top H_j\, g_j}{\sum_{j=1}^{n} \|g_j\|^2}, \tag{6}$$

where g_j and H_j are the gradient and Hessian of the loss on mini-batch j, over $n = 8$ random training mini-batches of the training batch size (a fresh random stream at every measured step). For mini-batch SGD without momentum, Andreyev and Beneventano (2024) find that BS stabilizes at $2/\eta$. We measure it for every optimizer and compare it with $(2+2\mu)/\eta_s$ for SGD.

Muon. We have no stability threshold for Muon, and report its $\lambda_{\max}$ and batch sharpness without one. Its update on a hidden matrix is orthogonalized, with all singular values near one whatever the size of the gradient, so the quadratic argument above, where the step scales with the gradient, does not carry over directly. This is a limitation. What we can test is the part of the model that Muon leaves to its auxiliary AdamW (Section 4.4).

One-step loss change. $\Delta L(s) = \ell_X(s) - \ell_X(s+1)$ on the training probe, used in the break-even analysis.

Edge of stability of the Adam-managed block.† Muon trains the embeddings, head and gains with an auxiliary AdamW; write a for these coordinates. At the fork checkpoints of the $d = 0$ runs (Table 6), which hold the optimizer states, we compute the top eigenvalue of $P_a^{-1/2} H_{aa} P_a^{-1/2}$, where H_{aa} is the block of the Hessian on $\mathcal{H}$ restricted to a and P_a the preconditioner P defined above, built from the state of the run's own Adam for these coordinates (Muon's auxiliary AdamW, or AdamW itself), by the same Lanczos procedure. We compare it with $(2+2\beta_1)/((1-\beta_1)\eta_s)$, with η and β_1 of that Adam. This treats the block as if the rest of the model were frozen, so it is a check of the auxiliary AdamW on its own, not a threshold for Muon.

What coincides with the split. For a sharpness measure $\mathcal{Q}$ with threshold Θ, the measured steps at the edge of stability are $\mathcal{E}(X) = \{s \in \mathcal{M} : \mathcal{Q}_X(s) \geq (1-\delta)\,\Theta_X(s)\}$ with $\delta = 0.1$. We report the onset $s_{\mathrm{EoS}}(X) = \min \mathcal{E}(X)$, the last step of $\mathcal{E}(X)$, and the fraction $|\mathcal{E}(X)|/|\mathcal{M}|$ of measured steps at the edge of stability, for $\lambda_{\max}$ (SGD) and the preconditioned sharpness (AdamW), and the ratio $\mathcal{Q}_X(s)/\Theta_X(s)$ at the steps t^*_X and $t^*_{\mathrm{p},X}$. Since $\mathcal{M}$ is partly log-spaced, the fraction weights early training more than a fraction of steps would. Batch sharpness is shown in the figures but has no onset computed. We compare these with t^*_X and $t^*_{\mathrm{p},X}$, with the episodes, with the end of warmup w, and with the loss levels across settings.

4.5 Swaps with warm optimizer states

Shadow states. During each $d = 0$ run of optimizer A, the other two optimizers are stepped on the same mini-batch gradients with learning rate 0. The state updates of SGD momentum, AdamW and Muon do not depend on the learning rate, and with learning rate 0 neither the update nor the decoupled weight decay moves the parameters; so their momenta, second moments and step counts evolve as if they were training, while the parameters follow A alone. (Our copy of Muon does not overwrite the gradient in place, so a shadow optimizer stepping after it sees the same gradient.)

Forks. At fork steps s (nine per setting, Table 6), the $d = 0$ run saves θ_s and the states of all three optimizers. The fork $A \to B$ at s loads θ_s and B's shadow state, keeps B's own hyperparameters and learning rate, and continues from s to T on the same schedule φ and the same batches. All six ordered pairs are forked at all nine steps. The control "A continued with the same batches" is the parent run itself. In addition, A is forked into A with data order 1 at the same steps, the control for mode connectivity (Section 4.6). This gives 81 forks per language-model setting. Forks log predictions and measurements like their parents.

Where a fork lands. Let F be the final model of the fork $A \to B$ at s, A the final model of the parent A_0, and B the final model of the pure run B_0 (from initialization, same batches). With D either distance of Section 4.1, the landing coordinate is

$$r = \frac{D(F, A)}{D(F, A) + D(F, B)}, \tag{7}$$

which is 0 at A's function and 1 at B's. It is a ratio of distances, not a projection: a fork far from both ends has r near $1/2$, so we report $D(F, A)$, $D(F, B)$ and $D(A, B)$ with it. We also give $D(F, B)$ over the final data-order floor of B, and $D(F, A)$ over that of $A^\dagger$: a fork within one floor of a pure run is as close to it as another data order is. For each function metric ϕ of Section 4.8 we also report the shift $(\phi(F) - \phi(A))/(\phi(B) - \phi(A))$, the fraction of the gap between the pure runs that the swap closes. If the function is decided at the split, forks with $s < t^*_A$ should end near $r = 1$ and forks with $s > t^*_A$ near $r = 0$. Since t^* turns out to be degenerate (Section 4.2), the time to compare fork steps with is the end of the early episode.

4.6 Linear mode connectivity

For two final weight vectors θ_1, θ_2 of the same architecture let $L(\alpha)$ be the mean cross-entropy of $(1 - \alpha)\theta_1 + \alpha\theta_2$ (every floating-point entry of the state interpolated) on the 64 sequences of the training probe. The barrier is

$$\mathrm{Bar}(\theta_1, \theta_2) = \max_{\alpha \in \{0, 0.1, \ldots, 1\}} \big[L(\alpha) - (1 - \alpha)L(0) - \alpha L(1)\big], \tag{8}$$

a loss version of the instability of Frankle et al. (2020). We compute it between the parent A_0 and each of its forks, as a function of the fork step: the forks $A \to A$ with data order 1 reproduce the instability analysis of Frankle et al. (2020) for each optimizer, and the forks $A \to B$ ask whether a change of optimizer at s leaves the solution in the same basin.

4.7 Anatomy of the top eigenvector

Partition the parameters into blocks: *emb* (token and position embeddings); for each layer i, qk_i (W_Q, W_K), ov_i (W_V and the attention output W_O) and mlp_i (both MLP matrices); *norms* (all LayerNorm gains); and *head*. For the CNN each layer is a block. Let v be the unit top eigenvector of H returned by Lanczos, and v_b the vector equal to v on the coordinates of block b and zero elsewhere. Since the blocks partition the coordinates, $v = \sum_b v_b$ and $\sum_b \|v_b\|^2 = 1$, and expanding the quadratic form gives

$$\lambda_{\max} = v^\top H v = \sum_b s_b + \chi, \qquad s_b = v_b^\top H_{bb} v_b, \qquad \chi = \sum_{b \neq c} v_b^\top H_{bc} v_c, \tag{9}$$

where H_{bc} is the block of H with rows in b and columns in c (and v_b is read as its restriction to block b where needed). We record, for each block, its share of the direction $\|v_b\|^2$ and its self-curvature s_b, and the cross-block remainder $\chi = v^\top H v - \sum_b s_b$, with $v^\top H v$ from one more Hessian-vector product (it equals the Lanczos estimate of $\lambda_{\max}$ up to the Lanczos error). Both s_b and χ can be negative, and $s_b \leq \lambda_{\max}(H_{bb})\|v_b\|^2$: a block carries a large part of the sharpness only if the direction lies in it and its own diagonal block is curved along it, while a large χ means the sharp direction comes from interactions between blocks. The split of the project plan, $v_b^\top (Hv)_b$, equals $\lambda_{\max}\|v_b\|^2$ for an exact eigenvector and so repeats the direction share; this is why we use (9) instead. For AdamW runs we repeat†

the decomposition for $P^{-1/2}HP^{-1/2}$ and its top eigenvector, whose diagonal blocks are $P_b^{-1/2}H_{bb}P_b^{-1/2}$. Figures sum blocks by type (all *qk*, all *ov*, all *mlp*) or by layer.

4.8 Function metrics

Offline, on saved checkpoints, we compute three groups of metrics at fixed validation points (definitions in Section B.1). *Input derivatives* of the loss as a function of the input (the token embeddings for the language models), from first to third order: gradient norm, Jacobian norm of the softmax output (Novak et al., 2018), Frobenius norm and top eigenvalue of the input Hessian (Moosavi-Dezfooli et al., 2019), level-set curvature (Srinivas et al., 2022), and three third-order quantities. *Weight norms*: those linear theory predicts each optimizer keeps small (ℓ_2, ℓ_∞, spectral), and norm-based measures compared by Jiang et al. (2020). *Accuracy and robustness*: Gaussian input noise for the CNN (Ford et al., 2019) and one-token substitution sensitivity (Hahn et al., 2021) for the language models. *Representations*†: linear CKA (Kornblith et al., 2019) between the final hidden states of two runs after each transformer block and after the final LayerNorm, on 64 training sequences, for pairs of pure runs that differ only in the optimizer, only in the data order, or in the initialization.

4.9 Break-even replication

We follow the protocol of Jastrzebski et al. (2020): plain SGD (no momentum) at a constant learning rate, without warmup. Besides $\lambda_{\max}$ (their λ_H^1) and ΔL, at each measured step we draw $L = 25$ mini-batch gradients $g_1, \dots, g_L$ and estimate the gradient covariance $\hat{K} = \frac{1}{L}\sum_i (g_i - \bar{g})(g_i - \bar{g})^\top$ through the eigendecomposition of its $L \times L$ Gram matrix. We record its top eigenvalue λ_K^1, its smallest non-zero eigenvalue λ_K^* ($\hat{K}$ has rank at most $L-1$), $\operatorname{Tr}\hat{K}$, and $\|g\|/\|g_5\|$, the mean over i of $\|g_i\|/\|\Pi_5 g_i\|$ with Π_5 the projection on the top five eigenvectors of $\hat{K}$. Prediction trajectories are embedded in two dimensions (PCA, then UMAP; Section B).

Runs. CIFAR-10: learning rates 0.001, 0.01 and 0.1 at batch size 128 (39,000 steps, 100 epochs), and batch sizes† 30 and 300 at learning rate 0.05 (also 100 epochs), with $\hat{K}$ always from batch-128 gradients; two data orders each. Shakespeare and TinyStories-10M†: learning rates 0.03, 0.1 and 0.3 at the training batch size, with the setting's model and length; three data orders each. A rate of 1.0 diverged within 15 (Shakespeare) and 20 (TinyStories-10M) steps and was dropped. The split analysis of Section 4.2 applied to two learning rates instead of two optimizers measures the separation of trajectories in prediction space.

5 Results

5.1 Learning-rate sweeps

Figure 15 (Section D) shows the sweeps. In all three settings Muon reaches the lowest validation loss, then AdamW, then SGD: best final validation loss 1.305, 1.339 and 1.914 on TinyStories-10M and 1.181, 1.201 and 1.340 on TinyStories-30M (SGD at its best rate on data order 0; 1.957 and 1.370 at the rates we use), and best minimum validation loss 1.54, 1.56 and 2.44 on Shakespeare (2.50 for SGD at 0.05, about the loss at which all three optimizers pass the bigram model, Section 5.3). SGD at 1 and 3 diverged on both TinyStories models. Without dropout the Shakespeare runs overfit, Muon fastest: its validation loss is lowest near step 500 and ends at 2.84–2.88 in the main runs, against 1.54–1.55 for AdamW (Fig. 16).

5.2 Divergence curves and the split

Table 2 summarizes the split analysis for all pairs and Fig. 2 shows three of them. The same pattern holds in every language-model pair.

The optimizers differ from the first steps. The first exceedance t^* is the first or second level of the loss grid, reached at steps 1–8. At that point the floor is still near zero, while two optimizers have already taken different first steps.

An early episode, then back to the floor.† Each pair has one long early episode above the floor. It ends at loss 4.59–6.82 on TinyStories and 2.77–3.16 on Shakespeare, and the ratio peaks in it at 32–204. The ratio is large because the floor is near zero; in absolute terms the cross distance in the episode is at most 0.034–0.045 bits for AdamW–Muon

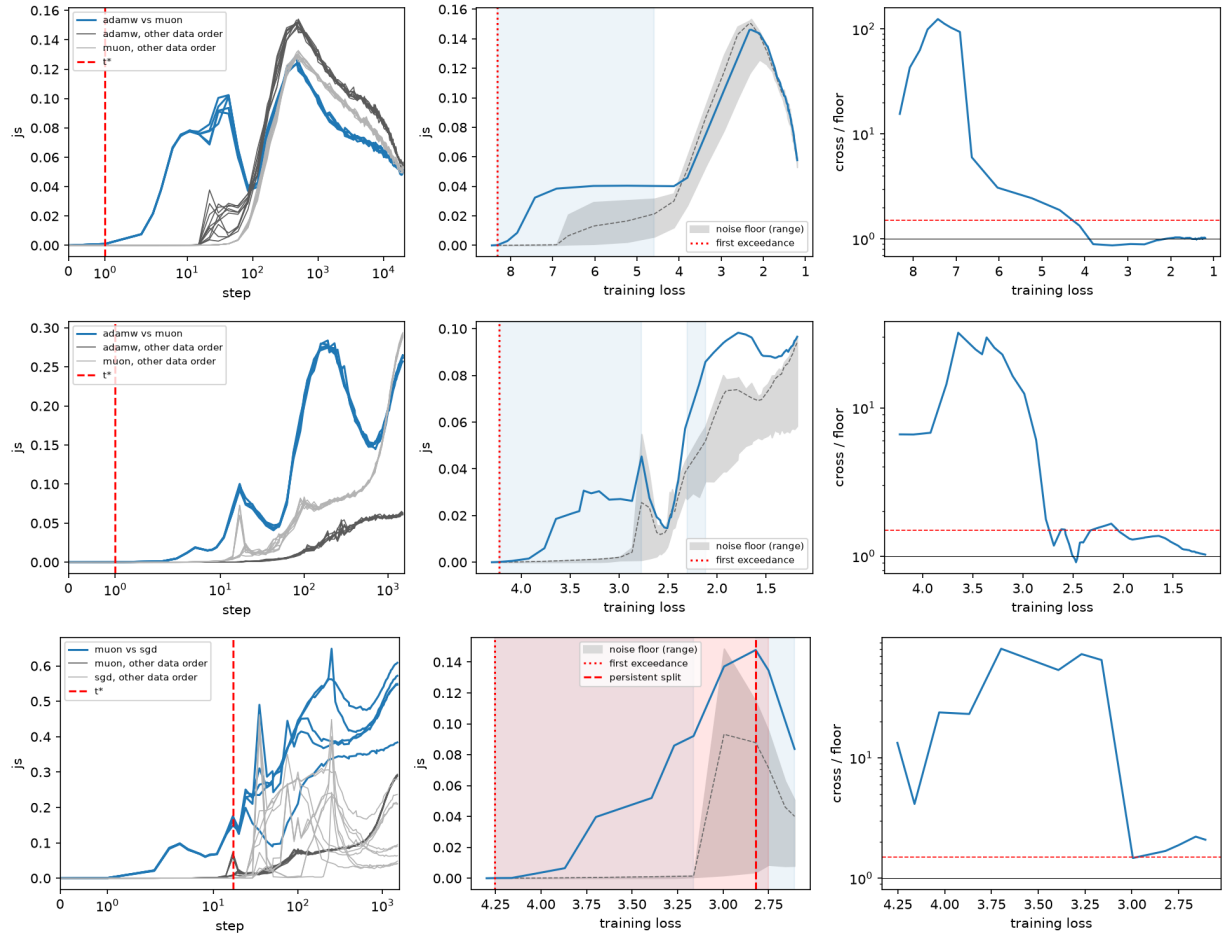


Figure 2: Function distance D_{JS} (bits) between optimizers. Rows: TinyStories-30M AdamW–Muon, Shakespeare AdamW–Muon, Shakespeare Muon–SGD (the only persistent split). Left: by step, for the 5 cross pairs (blue, same data order) and the 10 floor pairs of each optimizer (greys); the red line is the median step at which the runs of the first optimizer reach the persistent split, or the first exceedance t^* if there is none. Middle: loss-matched distance (blue) against training loss (axis reversed), with the range over floor pairs (grey band), the floor $F_{\mathcal{D}}$ (dashed), the episodes above the floor (shaded), t^* (dotted) and the persistent split (dashed, with its 95% interval shaded red). Right: cross distance over the floor, with the threshold $1 + m_{\times} = 1.5$ (red). The other pairs are in Figs. 17 and 18.

and 0.04–0.18 bits for pairs with SGD. After the episode the ratio returns to near 1 and stays there, except in the pairs with SGD on Shakespeare (below). For AdamW–Muon it lies in 0.95–1.38 (median 1.02) below loss 4.85 on TinyStories-10M, in 0.87–1.34 (median 1.01) below loss 4.59 on TinyStories-30M, and in 1.02–1.46 (median 1.11) below loss 2.12 on Shakespeare, after a second, short episode (2.30–2.12). At the lowest common loss all TinyStories pairs are at 0.99–1.03 times the floor. The largest absolute AdamW–Muon distances, 0.14–0.15 bits near loss 2.3 on TinyStories, occur where the floor is as large (middle panels).

One persistent split.† Only Muon–SGD on Shakespeare has a persistent split: a second episode from loss 2.82 to the end of the common grid at 2.61, found in 96.5% of resamples (95% interval 2.75–4.25). Muon reaches it at steps 14–17 and SGD at steps 43–550, depending on the data order. This pair tells us the least about how optimizers shape the function: the comparison stops at 2.61 because the slowest SGD run gets no further, SGD at 0.05 is barely stable there (loss spikes, Fig. 16; erratic floor, Fig. 2 bottom left), and four of its five runs end at training loss 2.32–2.58, about the bigram level (the fifth at 1.66). AdamW–SGD on Shakespeare is borderline: after its episode the ratio stays at 1.33–1.73, which is a persistent split in 34% of resamples but not in the full sample. On TinyStories SGD's own data-order noise is large (on TinyStories-30M one run ends at validation loss 1.61 against 1.37–1.39 for the others; on TinyStories-10M the runs end at 1.77–1.96), so "no split" against SGD means no larger than SGD's own data-order noise.

By step the picture differs. At equal steps (left panels) the optimizers look different for much longer, because they progress at different speeds: on TinyStories-30M Muon's loss stays flat for about the first ten steps and then falls faster

Table 2: Split analysis with D_{JS}. Grid: range of the common loss grid. t^*: first exceedance (5), found in 99.8% of resamples (all those with at least two distinct data orders) for every language-model pair but Matched–Muon, and in 52% for CIFAR-10 (two data orders), with a 95% interval equal to the point; steps: the steps at which A_0 and B_0 reach it. Episodes: loss ranges above the floor. Persistent: the persistent split t_{p}^* and the fraction of resamples in which one is found. Max and end: the ratio $\bar{c}_{\mathcal{D}}/F_{\mathcal{D}}$ at its maximum and at the last grid level. "Matched"†: Muon with its auxiliary AdamW matched to AdamW (Section 3). Break-even rows compare plain SGD at two learning rates.

Setting	Pair A–B	Grid	t^*	steps	Episodes†	Persistent†	Max	End
Shakespeare	AdamW–Muon	4.30–1.18	4.23	1 / 2	4.23–2.77, 2.30–2.12	none (0%)	32	1.02
	AdamW–SGD	4.30–2.62	4.23	1 / 1	4.23–3.10	none (34%)	98	1.69
	Muon–SGD	4.30–2.61	4.25	1 / 1	4.25–3.16, 2.82–2.61	**2.82** (96.5%)	80	2.09
	Matched–AdamW	4.30–1.19	4.26	1 / 1	4.26–2.85, 2.70–1.68	none (0%)	29	1.07
	Matched–Muon	4.30–0.23	none (4%)	–	none	none (0%)	1.28	0.98
TinyStories-10M	AdamW–Muon	8.39–1.35	8.26	1 / 7	8.26–4.85	none (0%)	111	0.99
	AdamW–SGD	8.39–2.04	8.26	1 / 3	8.26–6.32	none (0%)	162	1.03
	Muon–SGD	8.39–2.02	8.38	1 / 1	8.38–5.81	none (0%)	204	1.00
TinyStories-30M	AdamW–Muon	8.45–1.20	8.31	1 / 8	8.31–4.59	none (0%)	124	1.02
	AdamW–SGD	8.45–1.65	8.31	1 / 2	8.31–6.03	none (0%)	121	1.01
	Muon–SGD	8.45–1.65	8.44	1 / 1	8.44–6.82	none (0%)	200	1.00
CIFAR-10	η 0.001–0.01	2.30–1.09	2.27	4290 / 510	4 short within 2.27–1.13	none (0%)	19	1.47
	η 0.01–0.1	2.30–0.0004	0.50	11310 / 2730	0.50–0.41	none (0%)	1.75	1.24
Shakespeare	η 0.1–0.3	4.30–3.09	none (0%)	–	none	none (0%)	1.17	0.72

than AdamW's, and on Shakespeare Muon memorizes the training set while AdamW does not (Fig. 16). Section 5.3 separates the two effects.

Robustness.† We repeated the estimate for every pair and setting with D_{dis} instead of D_{JS}, and with D_{JS} at $m_\times \in \{0.25, 1\}$, $k \in \{2, 5\}$ and $\tau \in \{0.01, 0.05\}$, one change at a time (seven variants). On both TinyStories models no variant finds a persistent split for any pair, nor does any variant for AdamW and Muon on Shakespeare. The one persistent split, Muon against SGD on Shakespeare, is fragile: it is found with $k = 2$ (99% of resamples), $m_\times = 0.25$ and both tolerances, but not at the point estimate with D_{dis}, with $m_\times = 1$ (26% of resamples) or with $k = 5$ (42%). We read it as SGD ending slightly off the others where it stalls, not as a clear split.

5.3 Where the optimizers' routes meet†

Optimizer against data order. On TinyStories-30M (Fig. 3, top left), from step 332 on, the AdamW–Muon distance on the same data order is at or below both optimizers' floors. At step 20,000 it is 0.051 bits, against floors of 0.055 (AdamW) and 0.051 (Muon) and 0.056 for AdamW and Muon on different data orders: at the end of training, replacing AdamW by Muon changes the function no more than reshuffling the data. Sharing the data order does keep the two optimizers closer: the same-data distance is 20–32% below the other-data distance between steps 119 and 10,000, and 8% below at the end. Against SGD, the same-data distances at the end are 0.081 (AdamW) and 0.082 bits (Muon), above the median floors (0.051–0.063), and within 5% of the other-data distances; at equal steps this includes SGD being slower (final validation loss 1.37–1.39, one run 1.61, against 1.18–1.20 for AdamW and Muon). On Shakespeare (Fig. 19) the optimizer dominates at equal steps: at step 1500 the AdamW–Muon distance is 0.26 bits, four times AdamW's floor (0.06) but below Muon's own floor (0.29), which grows as Muon memorizes; same and other data orders give the same distance (0.263 and 0.264).

N-gram stages. On Shakespeare (Fig. 3, bottom) the three optimizers pass close to the bigram model at nearly the same training loss: the minimum distance to it is 0.036 (AdamW), 0.041 (Muon) and 0.041 bits (SGD), at loss 2.46, 2.47 and 2.48. At loss 2.47 the loss-matched AdamW–Muon ratio is 0.91. Before that the routes differ: SGD passes through the unigram model (0.001 bits at loss 3.09) and AdamW near it (0.054 at 3.20), while Muon stays further away (0.111 at 2.99). On TinyStories (Fig. 19) the optimizers come closest to the bigram model at loss 3.7–4.0, less closely than on Shakespeare (0.14–0.16 bits for AdamW and Muon, 0.20 for SGD); the AdamW–Muon ratio stays below 1.2 from loss 3.69 (TinyStories-10M) and 3.80 (TinyStories-30M) on. The early episodes therefore cover the stage before the bigram statistics are learned, where the optimizers take different routes, and end around the stage at which they all approximate the same n-gram solution.

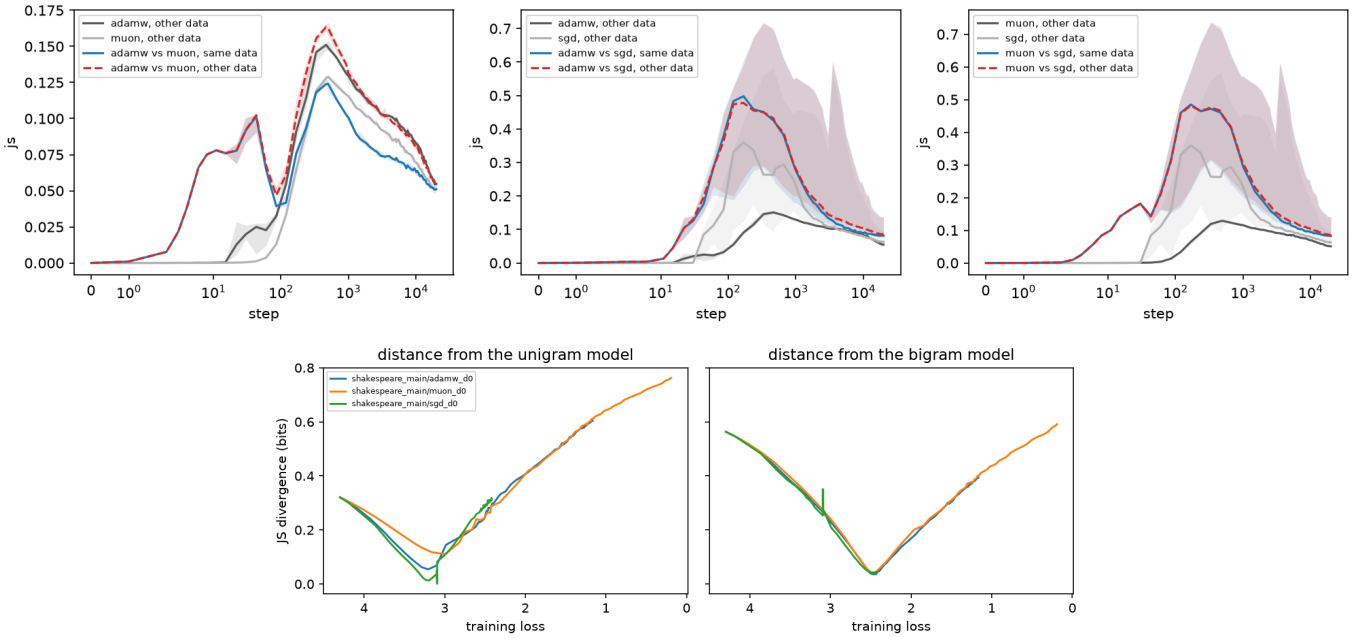


Figure 3: Top: optimizer against data order at equal steps on TinyStories-30M (Section 4.3): other optimizer on the same data order (blue) and on another data order (red, dashed), and the two floors (greys); median over the 5 data orders, range shaded. Bottom: D_{JS} of the $d = 0$ runs on Shakespeare from the unigram (left) and bigram (right) models of the training data, against training loss. Shakespeare decomposition and TinyStories n-gram distances in Fig. 19.

5.4 What coincides with the changes in distance

SGD reaches its threshold $(2 + 2\mu)/\eta_s$ only early in training, around and after the end of warmup (Fig. 4; Table 9 in Section D): in the $d = 0$ runs $\lambda_{\max}$ is within 10% of it, or above, only between steps 21 and 120, 62 and 1500, and 60 and 658, for warmups of 100, 200 and 400 steps, and at 2–16% of the measured steps over all data orders. After that, $\lambda_{\max}$ stays below a threshold that rises as the learning rate decays. On Shakespeare the SGD runs have large transient spikes (maximum ratio 7.9–956 over the data orders), matching their loss spikes. Batch sharpness stays below $\lambda_{\max}$, and so below the threshold, apart from early transients (Fig. 20).

AdamW. The preconditioned sharpness is above its threshold at the first measured steps (ratio 3 to 21 at step 1), when the second-moment estimate rests on one or a few gradients, so $s_{\mathrm{EoS}} = 1$ in every run. After a dip it returns to the threshold or above (on TinyStories-30M with spikes to 10–80 times it around steps 500–1000) and stays there for long stretches, until the cosine decay raises the threshold away from it: the ratio is at least 0.9 at measured steps up to 425–600 of 1500 on Shakespeare, 4500–9500 of 10,000 on TinyStories-10M and 9000–19,000 of 20,000 on TinyStories-30M (11–16%, 50–65% and 40–69% of the measured steps). This is the adaptive edge of stability of Cohen et al. (2022).

Muon. Without a threshold we only note that its $\lambda_{\max}$ fluctuates widely on Shakespeare (between about 10 and 400) and is smooth on TinyStories (Fig. 20).

Coincidences. No edge-of-stability event explains the changes in distance. The first exceedance, at steps 1–8, comes before the end of warmup and before SGD's onset; it coincides only with AdamW's onset at step 1, which reflects the preconditioner of the first steps rather than an edge of stability. The early episodes are short in steps: the $d = 0$ runs reach the loss at which the episode ends at steps 9–85 (221 for SGD in the TinyStories-10M Muon–SGD pair), within warmup in every pair but that one. For the pairs with SGD this is at SGD steps 20–45 (and 221), in the same range as SGD's onset in the $d = 0$ runs (steps 21, 62 and 60). But the AdamW–Muon pairs return to the floor in the same way without SGD, and AdamW, which reaches that loss at steps 29, 45 and 60, stays near its threshold long after the distance has returned to the floor. At the one persistent split (Muon–SGD, Shakespeare), each SGD run is below its threshold when it reaches loss 2.82 (ratio 0.18–0.80), and Muon's auxiliary AdamW is far below its own (ratio 0.02 at step 25, the first fork checkpoint after Muon reaches 2.82). As a loss level the end of the early episode is similar across the two TinyStories sizes (AdamW–Muon: 4.85 and 4.59) and close to the bigram stage (Section 5.3), which suggests describing it as a stage of learning rather than a step.

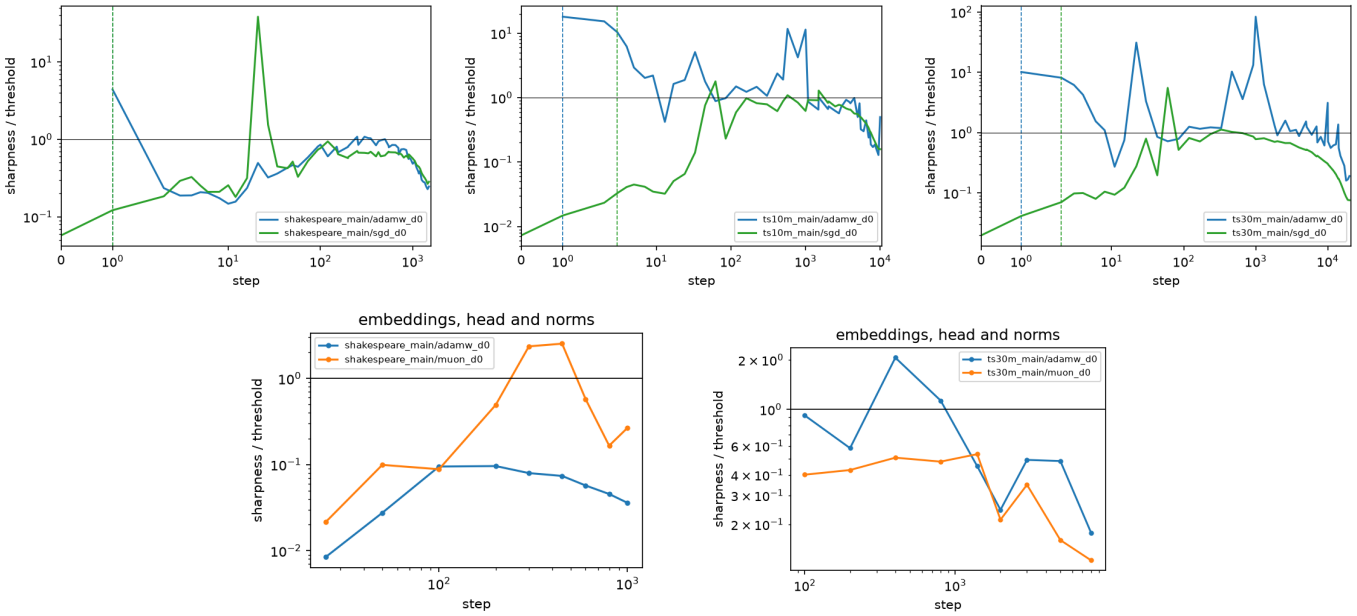


Figure 4: Top: sharpness over its stability threshold for the $d = 0$ runs on Shakespeare, TinyStories-10M and TinyStories-30M: $\lambda_{\max}\,\eta_s/(2+2\mu)$ for SGD and the preconditioned sharpness times $(1-\beta_1)\eta_s/(2+2\beta_1)$ for AdamW; 1 is the edge of stability, dashed lines the first exceedance t^*_X of the AdamW–SGD pair. Bottom†: the same ratio for the embeddings, head and gains alone (Section 4.4), at the fork checkpoints of AdamW and Muon, on Shakespeare (left) and TinyStories-30M (right). Raw sharpness, batch sharpness and thresholds in Fig. 20.

The block left to Muon's auxiliary AdamW.† On Shakespeare (Fig. 4, bottom left), the embeddings, head and gains of Muon exceed their threshold at the fork checkpoints at steps 300 and 450 (ratio 2.35 and 2.53) and are below it at the other seven (at most 0.57), while in the AdamW run the same block stays at most 0.10 times its threshold. These steps come just before Muon's validation loss starts to rise, and in this period Muon's top Hessian eigenvector lies almost entirely in the embeddings (Section 5.6). On TinyStories-30M it is the other way round: AdamW's block exceeds its threshold at steps 400 and 800 (2.06 and 1.13) and Muon's stays at most 0.53. In both settings the block that overshoots is trained by the Adam with the larger learning rate on these parameters (Shakespeare: Muon's 10^{-3} against AdamW's $3 \cdot 10^{-4}$; TinyStories-30M: AdamW's $2 \cdot 10^{-3}$ against Muon's $1.5 \cdot 10^{-3}$), so the learning rate plays a role; the control below shows that it is not the whole story.

Control.† Matching Muon's auxiliary AdamW to AdamW (Section 3) does not remove the memorization: matched Muon still reaches training loss 0.22 with validation loss 2.5, against 1.55 for AdamW, so the memorization comes from Muon's updates of the hidden matrices. With the same Adam, learning rate and threshold on the embeddings, head and gains as AdamW, matched Muon's block still reaches its threshold (ratio 1.14 and 1.08 at steps 600 and 1000, against at most 0.10 for AdamW; Fig. 25), later and less far than with Muon's default 10^{-3} (2.35 and 2.53 at steps 300 and 450). Its raw sharpness on this block reaches 236 and 579 at steps 600 and 1000, against at most 61 for AdamW. So Muon's updates of the hidden matrices make the embeddings sharper, and the larger learning rate of the default auxiliary AdamW brings the overshoot forward. In function space matched Muon is indistinguishable from Muon (first exceedance in 4% of resamples, ratio at most 1.28; Table 2), and against AdamW it behaves as Muon does: two episodes (4.26–2.85 and 2.70–1.68), no persistent split, and a ratio of 1.07 at the lowest common loss (Fig. 17).

5.5 Swaps and mode connectivity

Swaps. On TinyStories-10M (Fig. 5, top) the forks fall into three groups. *AdamW↔Muon*: at every fork step from 50 to 4000 the fork ends within the floor of the pure run of the new optimizer (0.86–0.95 floors) and, increasingly with the fork step, also of the parent's (0.92 floors at step 50, 0.20 at step 4000), so r falls from 0.5 to 0.18. Since the two pure runs are themselves within the floor of each other at the end (Section 5.3), these swaps cannot reveal an imprint of either optimizer. *SGD→AdamW and SGD→Muon* land on the new optimizer (0.91–1.08 floors from its pure run) at every fork step up to 4000, and stay 1.7–2.0 floors from pure SGD: 4000 steps of SGD leave no trace that AdamW or Muon do not erase. *AdamW→SGD and Muon→SGD* land on pure SGD when forked by step 200 (0.91–1.01 floors); later forks end further from it (1.04–1.42 floors at steps 400–1000), and from step 1500 on closer to the parent than to

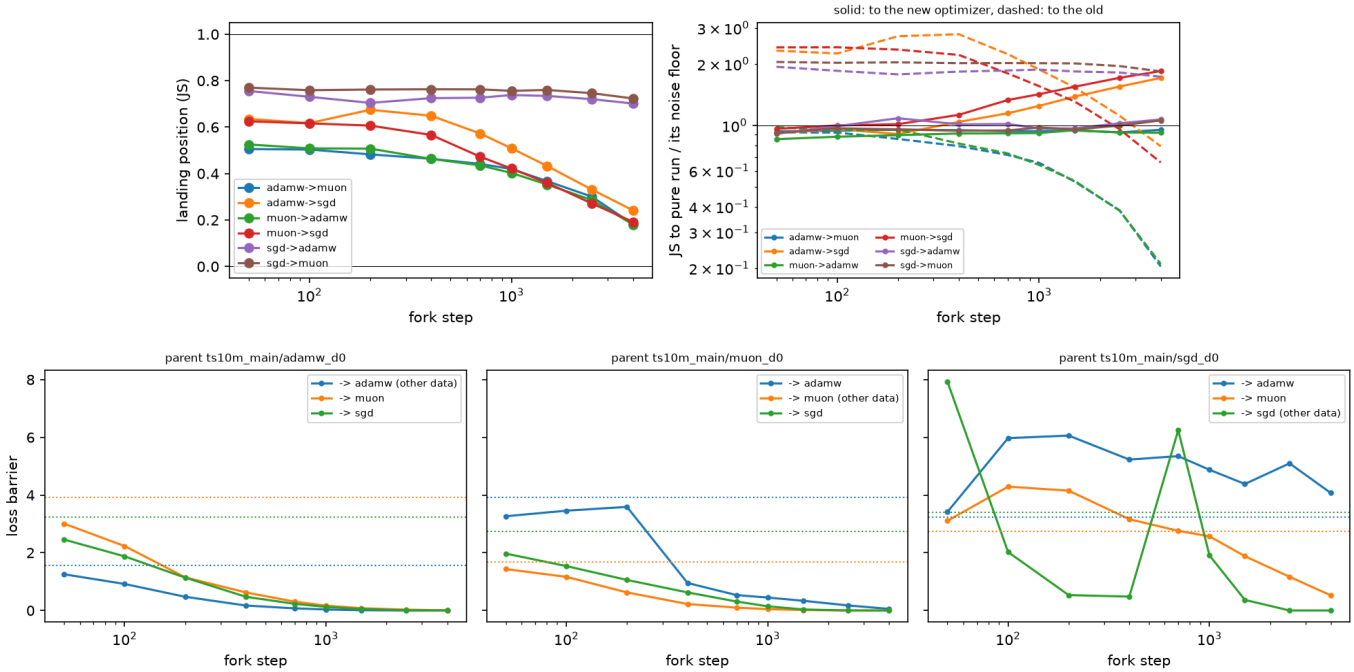


Figure 5: Swaps and mode connectivity on TinyStories-10M. Top left: landing coordinate r (7) with D_{JS} against the fork step, for the six ordered pairs; 0 is the final function of the parent (optimizer kept), 1 that of the pure run of the new optimizer (same batches). Top right†: the fork's distance to the pure run of the new optimizer (solid) and of the old one (dashed), each over that pure run's final data-order floor. Bottom: barrier (8) between the final weights of each $d = 0$ parent and of its forks against the fork step; "other data" are the forks that keep the optimizer and change the data order. Dotted†: barrier to the pure runs of the other optimizers and to the parent's optimizer with data order 1, which share only the initialization.

pure SGD (0.66–0.79 floors from the parent and 1.7–1.8 from pure SGD at step 4000): SGD at its stable rate is too slow to undo what the adaptive optimizer has already learned. In function space the last optimizer decides the endpoint whenever it is fast enough to do so, and none of the forks keeps an imprint of an early phase that the later optimizer could have reached on its own. Nearly all fork steps lie after the end of the early episodes (steps 9–85 in the $d = 0$ runs, Section 5.4), and r changes smoothly across them, so the end of the episode does not decide the endpoint either.

On Shakespeare (Fig. 28) the same rule holds, with Muon as the fast optimizer. AdamW→Muon and SGD→Muon land on pure Muon (0.93–1.05 floors) at every fork step from 25 to 1000. The reverse swaps do not land on the slower optimizer: from step 50 on, forks from Muon into SGD stay with Muon (0.43–1.03 of Muon's floor, 5.7–9.0 of SGD's), and forks from Muon into AdamW end 1.2 floors from pure AdamW at step 25 but 3.2 at step 1000. Muon's floor is large here (0.29 bits, against 0.06 for AdamW), as its runs memorize different things, so ending within it is a weak statement. Forks from SGD into AdamW keep a mark of SGD from step 50 on (3.2–5.7 floors from pure AdamW): AdamW at $3 \cdot 10^{-4}$ does not undo the early, unstable phase of SGD on Shakespeare in the remaining steps. Forks from AdamW into SGD are erratic (r between 0.01 and 0.93), like SGD's own runs there. In both settings, then, the optimizer that trains last decides the function when it is fast enough to undo the earlier optimizer's work in the remaining steps: on TinyStories whenever it is the faster one, and otherwise only for early forks.

Mode connectivity. Forks that keep the optimizer and change the data order (Fig. 5, bottom) become linearly connected to their parent at about the same time for AdamW and Muon: the barrier falls below 0.05 nats for forks from step 1000 (AdamW 0.035, Muon 0.046) and to 0 from step 2500, the stability point of Frankle et al. (2020) at 10–25% of training. For SGD the same barrier jumps between 0.4 and 7.9 up to step 1500, like its data-order noise (Section 5.2), and is 0 from step 2500. A change of optimizer needs a later fork to stay in the parent's basin: from the AdamW parent the Muon and SGD forks reach 0.03 and 0.01 nats only at step 2500, from the Muon parent the AdamW fork is still at 0.055 at step 4000, and from the SGD parent the AdamW fork is never connected (barrier 3.4–6.1 at every fork step) and the Muon fork only approaches it (0.53 at step 4000). Weight space thus keeps a mark of the optimizer that function space does not: the fork from AdamW to Muon at step 1000, which ends within the floor of both pure runs, has a barrier of 0.17 to its parent, against 0.035 for the fork that keeps AdamW and changes the data order. The pure runs, which share only the initialization, are not linearly connected for any pair (dotted lines); we compare them below.

On Shakespeare (Fig. 28, bottom) the forks that keep AdamW and change the data order are connected to their parent

from step 300 on (0.015, then 0), and even the pure AdamW runs with two data orders have barriers of only 0.11–0.17: at $3 \cdot 10^{-4}$ AdamW stays close to the initialization. Forks from AdamW into Muon or SGD keep barriers of 1.4–3.3 up to step 600, and those into Muon are still at 0.27 at step 1000; forks from SGD into Muon stay at 1.3–1.7 up to step 800. The exception is the Muon parent: its forks with another data order are connected only from step 800 (barrier 3.5 at step 25, 0.06 at 800), later than its forks into AdamW and SGD (0.9 and 0.7 at step 100, 0.01 at 800). Muon on Shakespeare is unstable to the data order for half of training, while it memorizes, and its forks into slower optimizers move the weights less after the fork. Apart from this case, a fork that changes the optimizer stays further from its parent in weight space than one that changes the data order, although it does not in function space.

Do swaps end in the new optimizer's basin?† A fork into B that ends at B's function need not end in the region of weight space of the pure B run. We computed the barrier between the final weights of each pure $d = 0$ run and of every fork into its optimizer (Fig. 31). On TinyStories-10M the forks into AdamW (2.9–8.0 nats) and into Muon (3.5–6.5) are further from the pure run than the same optimizer with another data order from the initialization (1.2–1.9 and 1.6–1.8 over ten pairs), at every fork step from 50 on, while the forks into SGD (2.4–3.4) are within SGD's own data-order barriers (0.1–4.1). The forks from AdamW into Muon, for instance, end within Muon's data-order floor in function space at every fork step, but 3.7–4.3 nats of barrier away from pure Muon. On Shakespeare the forks into AdamW (0.8–2.4) are far above AdamW's data-order barriers (0.11–0.17), while those into Muon (2.3–6.6) and SGD (0.9–2.3) are comparable to Muon's (3.6–6.6) and SGD's (1.0–3.2). As few as 50 steps of another optimizer at the start can move the endpoint further in weight space than a different order of all the data, while in function space they leave no trace.

TinyStories-30M. The larger model (Figs. 29 and 31) repeats these results, with one difference in the swaps into SGD. Forks between AdamW and Muon end within the floor of both pure runs at every fork step from 100 to 8000 (of 20,000), and r falls from 0.52 to 0.25. Forks from SGD land on the new optimizer (0.84–1.07 floors) at every fork step, but pure SGD ends only 1.1–1.4 floors from AdamW and Muon here, so these swaps say little. Forks from AdamW and Muon into SGD land near pure SGD up to step 3000 (0.92–1.21 floors), and only those at steps 5000 and 8000 end closer to the parent (0.57–1.00 floors from it) than to pure SGD, against step 1500 on TinyStories-10M: SGD's tuned rate is higher here (0.1 against 0.03), so it erases more. Forks that keep AdamW or Muon and change the data order are connected to their parent from step 2000 (0.022 and 0.040 nats, 0 from step 5000). Those that keep SGD have a barrier of only 0.14 at step 100 and below 0.06 from step 400, although the pure SGD runs with data orders 0 and 1 have a barrier of 8.8: SGD's sensitivity to the data order is confined to the first 100 steps. A change of optimizer again needs a later fork. From AdamW the SGD fork reaches 0.048 nats at step 3000, and the Muon fork 0.077 only at step 8000. From Muon the SGD fork is connected from step 3000 (0.068), but the AdamW fork is still at 0.19 at step 8000. From SGD the AdamW and Muon forks are never connected (0.69 and 0.29 at step 8000). The forks into Muon (4.4–5.7 nats) again end further from the pure run than another data order does (1.6–2.1), at every fork step. The forks into AdamW (3.5–18.8) do so only when forked at steps 200–400 from Muon and 200–5000 from SGD, since AdamW's own data orders are 3.3–6.8 nats apart here, and the forks into SGD (1.6–2.9, apart from 10.2 for Muon at step 100) stay within SGD's own (0.6–9.2).

Barriers between pure runs†. We computed the barrier between the final weights of every pair of pure runs that differ only in the optimizer or only in the data order, in full and with one group of parameters moved at a time (Fig. 6). In size, a change of optimizer is not consistently further than a change of data order. AdamW and Muon are further apart than two data orders of either on TinyStories-10M (3.5–4.1 nats against 1.2–1.9 and 1.6–1.8), but not on TinyStories-30M, where AdamW's data orders are 3.3–6.8 nats apart (AdamW–Muon 4.3–4.8), nor on Shakespeare, where Muon's are 3.6–6.6 apart (AdamW–Muon 2.1–2.7); SGD's data-order barriers vary so much (0.1–4.1 and 0.6–9.2 on TinyStories) that the pairs with SGD fall within them. Where the barrier sits does tell them apart. Between two data orders the single-layer barriers sit in the first block, up to 4.0 nats on average, and are at most 0.6 in any later block, or there are none at all (AdamW on Shakespeare, Muon on TinyStories-30M) although the full barrier is not small. Between AdamW and Muon on the same data they extend into the middle of the network: 0.8 and 0.4 nats in blocks 1–2 on TinyStories-10M, 2.3–0.5 in blocks 1–4 on TinyStories-30M and 1.1–1.9 in blocks 1–5 on Shakespeare. The pairs with SGD look like data orders, with the barrier mostly in the first block (and the embeddings on Shakespeare), and the head carries none (the loss is convex in the head). Two data orders, or SGD and an adaptive optimizer, thus end in solutions that differ in how the first block reads the embeddings, while AdamW and Muon also differ in how the middle blocks process them.

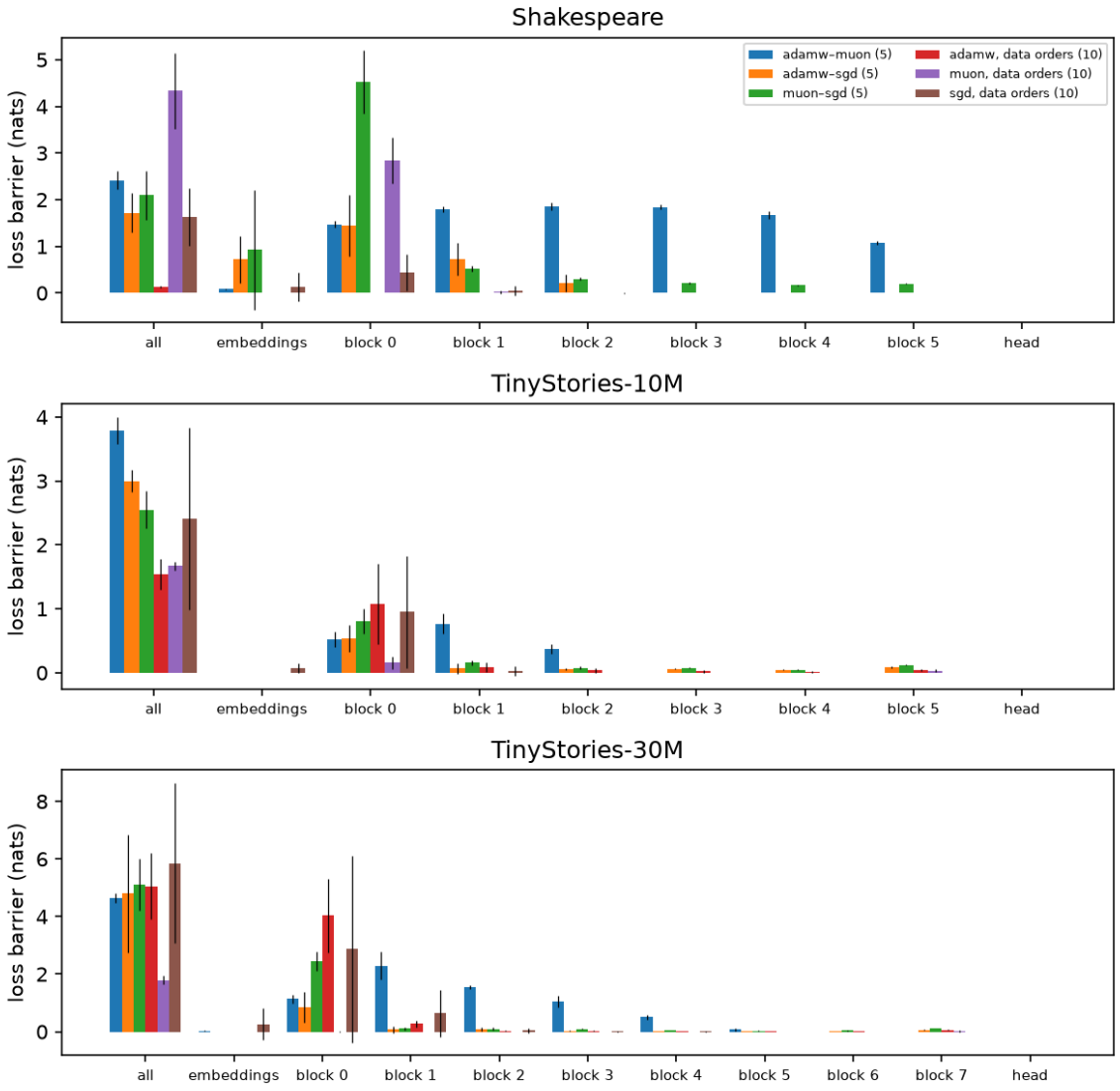


Figure 6: Loss barriers† between the final weights of pure runs, for every pair that differs only in the optimizer (5 per pair of optimizers) or only in the data order (10 per optimizer); mean and standard deviation over pairs. "All" is the ordinary barrier; for the other groups only that group of parameters moves along the straight line from the first run to the second, the rest stay at the first run's (Adilova et al., 2024).

5.6 Anatomy of the sharp direction

On TinyStories-30M (Fig. 7) the sharp direction differs by optimizer. Muon's lies mostly in the value–output matrices (about 60–70% of the direction) throughout training, with most of the rest in the MLP matrices. AdamW's lies in the head until about step 4000 and then largely in the gains (30–50%), with the value–output and MLP matrices. SGD's is spread over the MLP, value–output and head blocks. In all three the cross-block term χ is the largest part of the eigenvalue: about 75–85% for Muon and SGD throughout, and for AdamW about 25–40% while the head dominates and 60–70% after. The sharp direction is therefore not the top eigenvector of a single block but comes from curvature between blocks. On TinyStories-10M (Fig. 21) Muon's direction is again in the value–output matrices (60–70%), while AdamW's stays mostly in the head (60–75%). On Shakespeare Muon's direction lies almost entirely (85–97%) in the embeddings from step 200 on, with the embeddings' own curvature making up most of the eigenvalue: the sharp direction is in the block that Muon leaves to its auxiliary AdamW, including the period in which that block is over its threshold (Section 5.4). AdamW's direction on Shakespeare is spread over the MLP, value–output and embedding blocks, with about 70% of the eigenvalue from χ. The changes of dominant block (AdamW on TinyStories-30M, head to gains around step 4000–5000) happen long after the loss-matched distance between optimizers has returned to the floor. For AdamW the top eigenvector of the preconditioned†operator $P^{-1/2}HP^{-1/2}$, the one that sets its stability

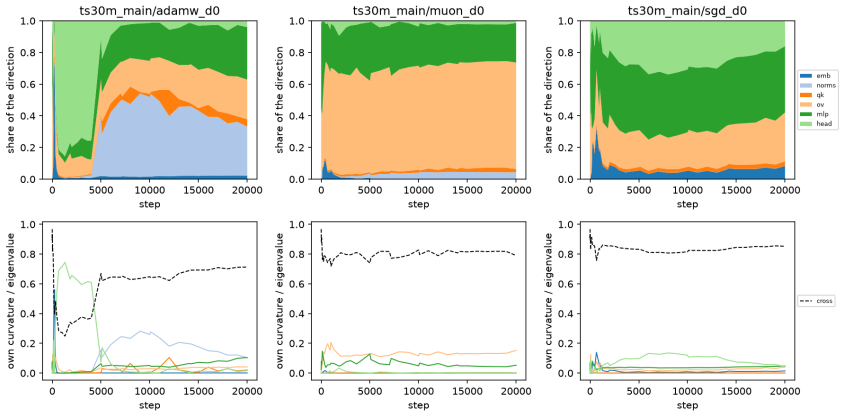


Figure 7: Anatomy of the top Hessian eigenvector of the $d = 0$ runs on TinyStories-30M (9). Top: share of the direction $\|v_b\|^2$ by block type. Bottom: each block type's own curvature s_b and the cross-block remainder χ (dashed), as fractions of the eigenvalue. Shakespeare and TinyStories-10M in Fig. 21.

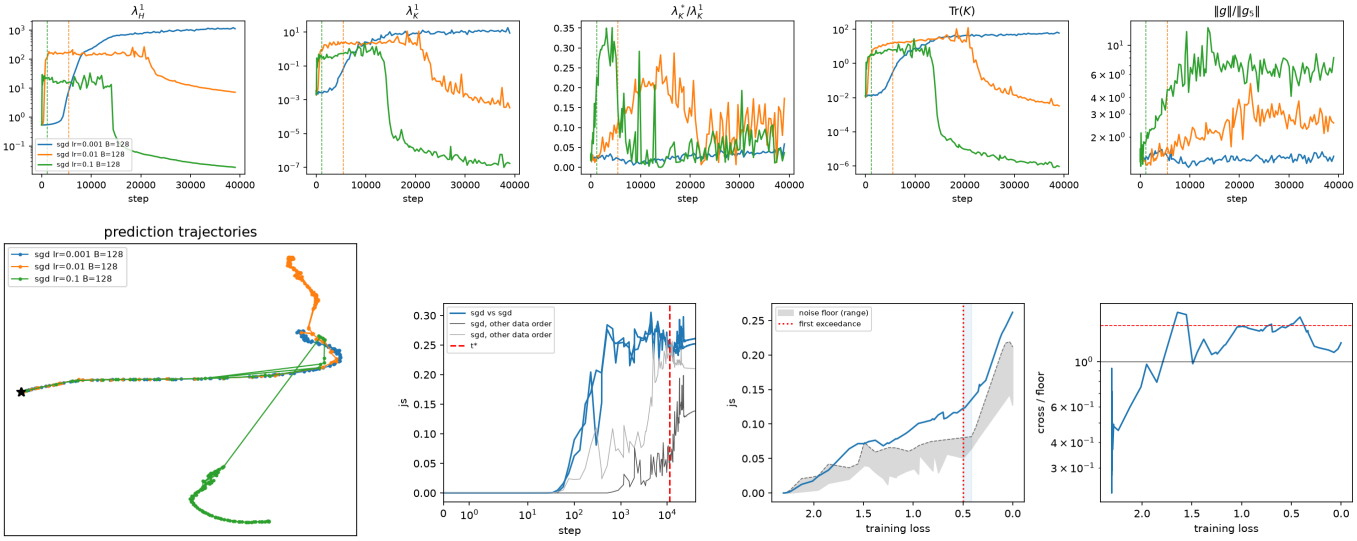


Figure 8: The break-even analysis of Jastrzebski et al. (2020) on CIFAR-10 with plain SGD at $\eta \in \{0.001, 0.01, 0.1\}$, batch 128. Top: λ_H^1, λ_K^1, λ_K^*/λ_K^1, Tr $\hat{K}$ and $\|g\|/\|g_5\|$ against step ($d = 0$). Bottom left: prediction trajectories (PCA then UMAP), starting at the star. Bottom right: the divergence figure of Fig. 2 for $\eta = 0.01$ against $\eta = 0.1$, with two data orders. Shakespeare in Fig. 23.

threshold, lies elsewhere than that of H (Fig. 22): mostly in the MLP matrices (40–60%), the attention matrices and the embeddings on both TinyStories sizes, rather than in the head or the gains, with short periods (the first 1000 steps and a few isolated measurements) in which it lies almost entirely in the embeddings and their own curvature makes up the eigenvalue. Outside those periods 80–95% of the preconditioned eigenvalue comes from curvature between blocks, even more than for H.

5.7 Break-even replication

CIFAR-10. The effects reported by Jastrzebski et al. (2020) replicate (Fig. 8). The larger the learning rate, the lower the sharpness in the early phase: λ_H^1 levels off at about 1000, 200 and 20 for $\eta = 0.001$, 0.01 and 0.1, that is at or below $2/\eta$, and the maximum of λ_K^1 decreases with the learning rate in the same way. The larger rates also give a better-conditioned gradient covariance (higher λ_K^*/λ_K^1) and a larger $\|g\|/\|g_5\|$. The relation between λ_H^1 and λ_K^1 and ΔL is in Fig. 23. In prediction space the three runs share a path from the initialization; $\eta = 0.1$ leaves it first, then $\eta = 0.01$. At equal steps the distance between $\eta = 0.01$ and 0.1 is far above the data-order floor (bottom right, left panel). At equal training loss, however, it is 1.0–1.75 times the floor at every loss below 1.8 (median 1.24); it exceeds 1.5 times the floor at three consecutive levels only once (loss 0.50–0.41) and there is no persistent split. For $\eta = 0.001$

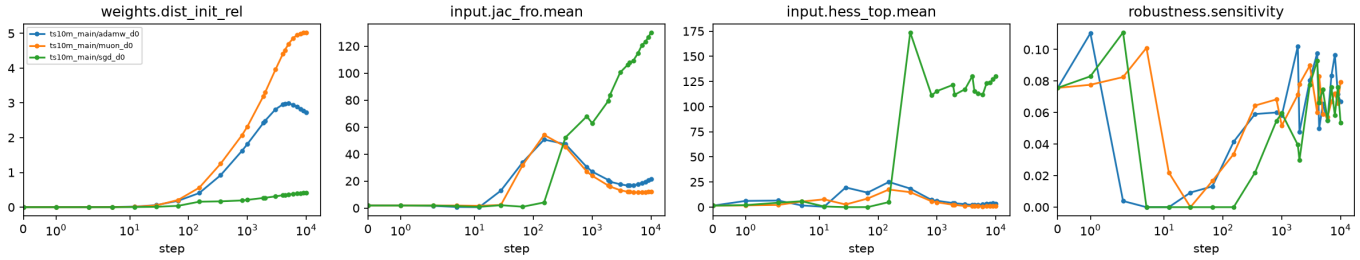


Figure 9: Function metrics over training for the $d = 0$ runs on TinyStories-10M: distance of the weights from the initialization relative to the initial norm, Frobenius norm of the input Jacobian of the softmax output, top eigenvalue of the input Hessian of the loss (inputs are the token embeddings), and one-token substitution sensitivity (Section B.1).

Table 3: Final function metrics on TinyStories, mean over the five data orders (standard deviations are small except for SGD's spectral norms and stable rank). "Hidden" are the matrices in the transformer blocks, the ones Muon updates; the stable rank is $\|W\|_F^2/\|W\|_2^2$.

	TinyStories-10M			TinyStories-30M		
	SGD	AdamW	Muon	SGD	AdamW	Muon
$\|\theta\|_2$	105	265	485	156	369	857
$\|\theta\|_\infty$	4.37	2.09	1.85	3.33	2.58	2.28
max $\|W\|_2$ (hidden)	3.5	12.4	14.1	7.6	15.0	22.9
$\|\theta\|_\infty/\|\theta\|_2$ ($\times 10^3$)	41.6	7.9	3.8	21.3	7.0	2.7
stable rank (hidden, mean)	56	41	75	56	47	90
$\|\theta - \theta_0\|/\|\theta_0\|$	0.40	2.72	5.04	0.61	2.83	6.56
input Jacobian (Frobenius)	131.7	21.6	12.5	72.7	15.2	7.8
input Hessian $\lambda_{\max}$	123.6	3.5	1.1	37.2	2.4	0.5
substitution sensitivity	0.070	0.082	0.067	0.076	0.060	0.075
top-1 accuracy	0.576	0.651	0.657	0.637	0.677	0.680

against 0.01 the ratio is 19 near the initialization, where the floor is near zero, and 0.57–2.08 below loss 1.5 (median 1.04), with four short episodes and no persistent split. As for the optimizers, most of the separation in time is one run being faster.

Language models.† On Shakespeare, plain SGD at 0.1 and 0.3 without warmup blows up in the first ten or so steps, where the distance between any two runs, floor pairs included, approaches 1 bit; the runs recover but reach only training loss 3.09 (the lowest common loss). At equal loss the distance between the two rates is at most 1.17 times the floor and no split is found (Table 2). λ_K^*/λ_K^1 rises over training at both rates (Fig. 23), and λ_H^1 is lower at 0.3 apart from spikes. At $\eta = 0.03$ the runs do not blow up; against 0.1 the ratio exceeds the floor in one episode (loss 3.74–3.28, at most 5.2 times the floor, in 68% of resamples) and is 0.90 at the lowest common loss, 3.13. On TinyStories-10M the three rates reproduce the sharpness and noise effects: λ_H^1 levels off at about 125, 35 and 20 for $\eta = 0.03$, 0.1 and 0.3, and λ_K^1 and $\operatorname{Tr}\hat{K}$ decrease with the learning rate; but λ_K^*/λ_K^1 is, if anything, higher at the smallest rate, the opposite of CIFAR-10. In prediction space 0.03 against 0.1 shows the pattern of the optimizer pairs: up to 151 times the floor at loss 7.4, one episode down to loss 5.14, and 0.97 times the floor (median) below loss 4, down to the lowest common loss 2.92 (Fig. 24). The rate 0.3 recovers slowly from its initial blow-up, so 0.1 against 0.3 is compared only down to loss 4.80 (one episode, 7.78–6.24, ratio 1.07 at the end).

Batch size.† On CIFAR-10 at $\eta = 0.05$, batch 30 against 300 also replicates the sharpness effect (the smaller batch levels off at a lower λ_H^1, about 12 against 50), and at equal loss the two batch sizes are 0.66–2.14 times the floor below loss 2 (median 1.22; at most 3.1 near the initialization), with no episode and no persistent split (Fig. 24).

5.8 Function metrics

The raw norms mostly measure how far each optimizer moves the weights (Table 3): Muon ends 5.0–6.6 initial norms away from the initialization, AdamW 2.7–2.8 and SGD 0.4–0.6, and $\|\theta\|_2$ and the largest spectral norm of the hidden matrices follow the same order. So SGD having the smallest ℓ_2 norm, and Muon the largest spectral norm, says little about implicit bias. The scale-free versions† match the predicted geometries in part. The ratio $\|\theta\|_\infty/\|\theta\|_2$, small when

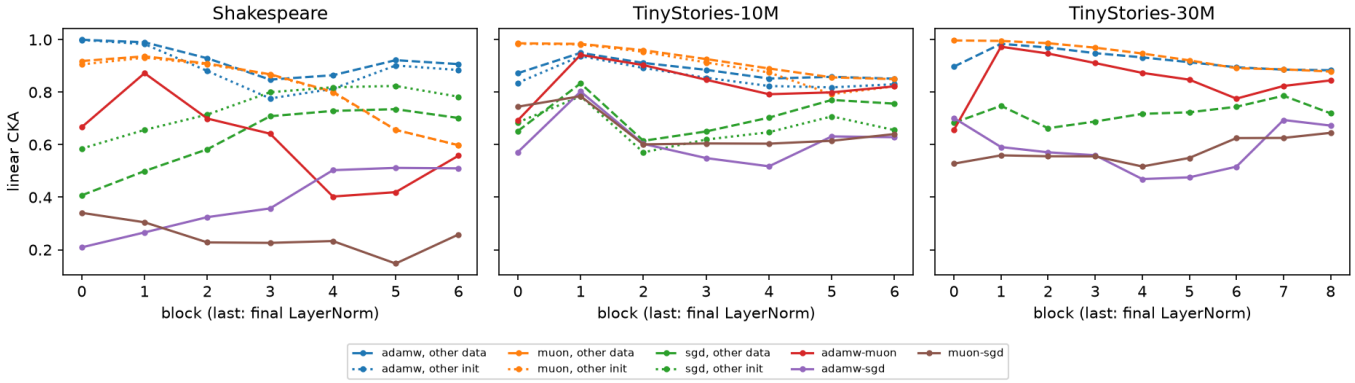


Figure 10: Linear CKA† between the final hidden states of two pure runs after each transformer block (the last point is the final LayerNorm), averaged over pairs. Solid: other optimizer, same initialization and data order; dashed: same optimizer, other data order; dotted: same optimizer, other initialization (not run on TinyStories-30M).

the entries are even, is 3–5 times smaller for AdamW than for SGD, as Adam's ℓ_∞ bias predicts (Zhang et al., 2024a); and the mean stable rank of the hidden matrices, large when their singular values are even, is highest for Muon (75 and 90 against 41–56), as its spectral bias predicts (Fan et al., 2025). Muon also has the smallest $\|\theta\|_\infty/\|\theta\|_2$. The weights therefore carry the geometry of their optimizer while the functions of AdamW and Muon are within the data-order floor of each other (Section 5.3), in line with the barriers of Section 5.5.

In input space SGD's function looks much sharper: its input Jacobian is 5–11 times and its top input-Hessian eigenvalue 15–110 times those of AdamW and Muon. The inputs are the token embeddings, which SGD moves least (Frobenius norm of the embedding matrix 27 for SGD against 97 for AdamW and 175 for Muon on TinyStories-10M), and the pre-LayerNorm transformer normalizes the residual stream, so derivatives with respect to the embeddings shrink as the embeddings grow; much of the difference is therefore one of scale. One-token substitution sensitivity, which does not depend on that scale, is 0.06–0.08 for all three; and the input Jacobian times the embedding norm is about the same for AdamW and Muon (2.1 and 2.2×10^3) and 1.7 times higher for SGD. Over training (Fig. 9) the input Jacobian of AdamW and Muon peaks at about step 200, the end of warmup, and then falls, while SGD's keeps rising.

On Shakespeare (Table 10) AdamW's learning rate of 3×10^{-4} moves the weights by only 0.25 initial norms, as little as SGD (0.36), and the two have the same $\|\theta\|_\infty/\|\theta\|_2$ (12.4 and 12.3×10^{-3}): the ℓ_∞ geometry only shows once the weights leave the initialization. Muon again has the largest stable rank (99 against 48 and 63), and, as it memorizes, twice to four times the substitution sensitivity of the others (0.19 against 0.10 and 0.05).

Representations†. Between weights and outputs sit the hidden states (Fig. 10). On TinyStories, AdamW and Muon trained on the same data have less similar representations than two data orders of either optimizer, but not by much outside the first block: linear CKA 0.69 and 0.66 after the first block against 0.87–1.0, and 0.78–0.97 against 0.85–0.98 after the others, 0.82–0.85 against 0.85–0.88 after the final LayerNorm. SGD's representations are not even reproducible across its own data orders (0.57–0.83), and pairs with SGD are lower still. On Shakespeare, where Muon memorizes, AdamW and Muon differ clearly (0.40–0.87). Another initialization moves the representations as much as another data order. So the optimizer's mark fades from the weights (their geometry, and barriers in the middle blocks) through the representations (a small CKA gap) to the outputs (none).

Off the data distribution†. The optimizers could agree on the data and differ elsewhere, where the data do not constrain the function and an implicit bias has room to act. We compared the final functions of the pure runs on the 32 validation sequences, on the same sequences with their tokens shuffled within each sequence, and on uniformly random tokens (Fig. 11). Off the distribution every pair of runs differs much more (on TinyStories, 0.20–0.25 bits on shuffled and 0.30–0.40 on random tokens, against 0.05–0.06 on validation), but the optimizer adds no more than the data order does: AdamW and Muon are 0.90–1.07 floors apart on every input type in all three settings. SGD, less trained at equal steps, is 1.2–2.7 floors from the adaptive optimizers on validation but only 1.05–1.8 off the distribution, so what separates it is how well it fits the data, not how it extrapolates.

Metrics under swaps. Figure 12 places each fork's final metrics (the scale-free ones added after the plan†) between those of pure A (0) and pure B (1). The weight metrics follow whichever optimizer moved the weights more, not the one that decides the function. Forks into SGD keep the old optimizer's $\|\theta\|_\infty/\|\theta\|_2$ and input Jacobian after switching at step 400 or later (shift ≤ 0.37), although at steps 400–1000 their function is within 1.0–1.4 floors of pure SGD (Fig. 5); Muon to SGD keeps Muon's stable rank at every fork step. Forks from SGD take the new optimizer's metrics, as

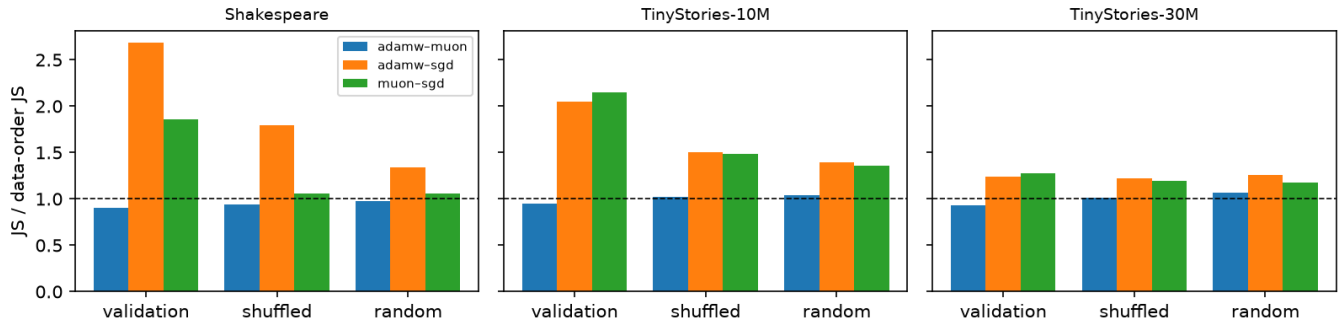


Figure 11: Distance D_{JS} off the data distribution† between the final functions of two optimizers (same initialization and data order), averaged over the five data orders, over the larger of the two optimizers' mean distance between data orders, on validation sequences, on the same sequences with shuffled tokens, and on uniformly random tokens.

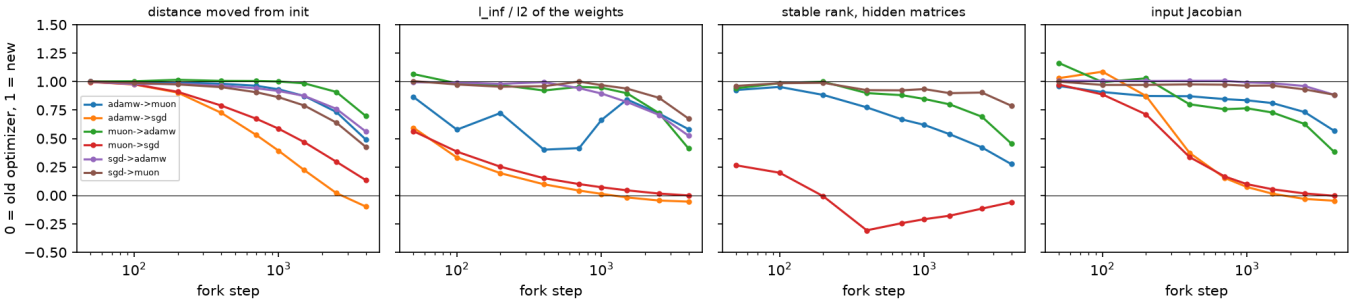


Figure 12: Final metrics of the forks on TinyStories-10M as a fraction of the gap between pure A (0) and pure B (1), against the fork step. Pairs whose pure gap is under three data-order standard deviations are left out (AdamW–SGD in stable rank).

they take its function. Between AdamW and Muon, whose functions cannot be told apart, the late forks are mixtures: switching at step 4000 goes only 0.28 (AdamW to Muon) and 0.45 (Muon to AdamW) of the way to the new optimizer's stable rank, and 0.4–0.6 of the way in $\|\theta\|_\infty/\|\theta\|_2$. The geometry of the endpoint records both optimizers and their order, while the function records only the faster of the two. On Shakespeare the same holds where the pure gap exceeds the noise, with Muon, which moves the weights furthest, setting the geometry (Fig. 30). On TinyStories-30M the picture is the same, from a later fork step for the swaps into SGD: switching at step 800 or later keeps the old optimizer's $\|\theta\|_\infty/\|\theta\|_2$ (shift ≤ 0.20), and at step 1400 or later its input Jacobian (≤ 0.19), while up to step 3000 the function of these forks is within 1.02–1.21 floors of pure SGD; AdamW and Muon swapped at step 8000 go 0.29 and 0.65 of the way in stable rank and 0.45 in $\|\theta\|_\infty/\|\theta\|_2$.

5.9 Further controls†

Second initialization. All other runs share one initialization, so the floor varies only the data order. We repeated the main runs of Shakespeare and TinyStories-10M from a second initialization (seed 1) with data orders 0 and 1, without curvature measurements. On Shakespeare (Fig. 13, top) the distance between the two initializations starts at 0.05 bits, falls to about 0.01 by step 10–100 and from about step 100 on equals the data-order floor for every optimizer; at the final step it is 0.068 against 0.063 (AdamW), 0.288 against 0.292 (Muon) and 0.053 against 0.067 (SGD). The split analysis at the second initialization, with two data orders, reproduces that at the first: AdamW–Muon has episodes at loss 4.18–2.86 and 2.41–2.02 (first initialization: 4.23–2.77 and 2.30–2.12) and no persistent split; AdamW–SGD 4.18–3.04 and 2.77–2.54 (first: 4.23–3.10), no persistent split; and Muon–SGD again has a persistent split, from loss 2.80 (first: 2.82). On TinyStories-10M (Fig. 26) the two distances also agree from about step 100 on, and at the final step the initialization, the data order and the change between AdamW and Muon all give the same distance: 0.054, 0.062 and 0.058 bits from AdamW, 0.055, 0.059 and 0.058 from Muon. The episodes are again close to those of the first initialization, AdamW–Muon 8.31–4.27 (first: 8.26–4.85), AdamW–SGD 8.31–6.21 (8.26–6.32) and Muon–SGD 8.43–6.78 (8.38–5.81), with no persistent split. Changing the initialization thus moves the function by as much as changing the data order, and the structure of the episodes does not depend on the initialization.

Rare and frequent tokens. Kunstner et al. (2024) attribute Adam's advantage over SGD on language models to the rare classes of a heavy-tailed vocabulary. We grouped the probe positions by the training frequency of their target token (Fig. 13, middle, for TinyStories-10M). SGD's loss gap is concentrated on the rare tokens: on TinyStories-10M it is 1.15 nats above AdamW on the rarest fifth and 0.12 on the most frequent (TinyStories-30M: 0.48 and 0.07). At the final step its distance from AdamW and Muon is also largest on the rare tokens (1.8–2.1 times the floor, against 1.4 on the most frequent), but at the lowest training loss both runs reach it is 0.93–1.03 times the floor in every bin, for every pair on TinyStories. The gap on rare tokens is a difference in speed, not in the function learned at equal loss.

A positive control. Since the detector finds no persistent split between AdamW and Muon, we checked that it finds one where theory predicts it. Logistic regression on linearly separable data ($d = 50$, $n = 200$, labels from a sparse teacher on 3 coordinates, 2000 test points), from zero initialization with batch 20, trained by SGD with momentum ($\eta = 0.1$) and by Adam ($\eta = 0.01$) for 10^5 steps with five data orders: in the limit gradient descent converges in direction to the ℓ_2 max-margin classifier (Soudry et al., 2018) and Adam to the ℓ_∞ one (Zhang et al., 2024a). The same loss-matched JS rule, on the test predictions, finds the ratio below 1.5 down to training loss 10^{-2} (at most 1.47) and a persistent split from loss $7.5 \cdot 10^{-4}$, with the ratio up to 3.3 (Fig. 13, bottom). The detector can see a split, and in the linear case the split appears near interpolation, far below the losses our language models reach. Neither optimizer is at its limit after 10^5 steps: Adam's normalized margins exceed SGD's in both norms (0.221 against 0.202 in ℓ_2, 0.377 against 0.341 in ℓ_∞).

Training into memorization. If the language models split only near interpolation, as the linear model does, our runs may simply stop too early. On Shakespeare we therefore trained AdamW and Muon for 8000 steps instead of 1500 (same learning rates, cosine schedule over the longer run, five data orders each), so that both memorize: AdamW's training loss falls to 0.09 and its validation loss rises to 4.6. The loss-matched ratio (Fig. 27; Fig. 1, top left, red) has a single early episode (loss 4.23–2.64) and then falls steadily to 1.0 at the lowest common loss, 0.09; with top-1 disagreement the picture is the same (episode 4.23–2.85, no persistent split). At the final step the two optimizers on the same data order are as far apart as two data orders of either (0.359 bits against floors of 0.355 and 0.324). Memorizing the training set, AdamW and Muon reach functions that differ on held-out text by a lot, but by no more than two data orders do.

Other learning rates. The comparisons above use one tuned learning rate per optimizer. The sweep runs (data order 0, same initialization) give the comparison at every other rate for free: each against the tuned run of every optimizer on the same data, at equal training loss, over the data-order floor of the tuned runs, summarized by the median below the bigram loss (Fig. 14). On TinyStories every rate that trains stably, a 27-fold range for AdamW, 8-fold for Muon and 3- to 10-fold for SGD, is within 0.80–1.11 floors of all three tuned optimizers, with one exception: SGD at 0.1 is 1.35 floors from tuned SGD at 0.03 on TinyStories-10M, further than from tuned AdamW or Muon. Only rates at the edge of stability (Muon at 0.08, SGD at 0.3 on TinyStories-10M) are 1.5–2.3 floors away, and higher ones diverge. On Shakespeare, where SGD does not get below the bigram loss at most rates, AdamW at 3–13 times its tuned rate is 1.5–1.6 floors from tuned AdamW but 1.03–1.06 from tuned Muon, since the floor of a cross comparison is the larger of the two optimizers' floors and Muon's is large (it memorizes). The floor of the tuned runs may not be the floor at other rates, so this is a check, not a second analysis; but it shows that the absence of a split on TinyStories is not a property of the chosen rates.

6 Discussion and limitations

Apart first, together later. The hypothesis of the title, that optimizers first follow one path and separate later, is not what we observe. At equal training loss the optimizers are most different at the start, while the data-order floor is still near zero, and the difference disappears around the stage at which all of them approximate the bigram statistics of the data. From there on, on TinyStories, two optimizers on the same data order are as close as one optimizer on two data orders, down to the lowest loss we reach. One reading is that the early phase is governed by the order in which simple statistics are picked up, which depends on the optimizer (on Shakespeare SGD and AdamW pass through the unigram model and Muon does not), while afterwards the function is set mostly by the data and the optimizer matters mainly through speed. This rests on comparing at equal loss: at equal steps all pairs look different for most of training, and on Shakespeare Muon's faster memorization dominates the comparison.

What the floor means. Being within the floor does not mean computing the same function: at the end of the TinyStories runs two data orders of AdamW or Muon differ by 0.051–0.062 bits. The finding is that changing the optimizer perturbs the learned function no more than changing the order of the data or, where we tested it, the initialization (Section 5.9), not that it does not perturb it. The asymptotic implicit-bias results (Gunasekar et al.,

2018; Zhang et al., 2024a; Fan et al., 2025) describe the limit of long training of linear models on separable data. Our TinyStories models see at most 1.2 epochs, and by then the loss-matched difference between optimizers is within the data-order noise; in the linear control the split appears only near interpolation. Nor do the swaps show an early decision: a fork ends at the function of the optimizer it switches to whenever that optimizer is fast enough to get there in the remaining steps (on TinyStories, whenever it is the faster one, at every fork step we tried), and keeps the mark of its parent only when the new optimizer is too slow to move away from it.

Function space and weight space. The mode-connectivity barriers tell a different story from the function distances. After a change of optimizer a fork needs a later fork step to stay linearly connected to its parent than after a change of data order, except for Muon on Shakespeare, which is itself unstable to the data order while it memorizes. Between pure runs the size of the barrier does not single out the optimizer (AdamW and Muon are further apart than two data orders only on TinyStories-10M), but its location does: the barrier between AdamW and Muon extends into the middle blocks, while that between two data orders lies in the first block, although their functions are as close. Different optimizers thus reach functions that are equivalent up to data-order noise through weights that differ beyond the first block, whereas data orders differ mainly in the first. The weights also carry the geometry each optimizer is biased towards: more even entries for AdamW than for SGD, and more even singular values for Muon (Section 5.8), and after a swap that geometry is a mixture weighted by how far each optimizer moved the weights, even when the function is entirely the second optimizer's. The hidden states sit in between: on TinyStories AdamW and Muon differ after the first block and little after the others. Part of the barriers may be scale: the optimizers move the weights by very different amounts (on TinyStories Muon's end 5–6.6 initial norms from the initialization, AdamW's 2.7–2.8 and SGD's 0.4–0.6), and interpolating between weights of different scales can raise the loss without any change in function.

Break-even point. On CIFAR-10 we reproduce the sharpness and gradient-noise effects of Jastrzebski et al. (2020), but the separation of trajectories between learning rates is mostly a difference in speed: at equal loss, 0.01 and 0.1 stay within 1.75 times the floor. Comparing at equal steps overstates how differently two training procedures shape the function.

Edge of stability. On TinyStories AdamW is at its adaptive edge of stability through much of training whether or not the optimizers differ, and SGD meets its threshold only early in training; neither explains the start or the end of the early episode, which ends within warmup in all pairs but one and in the same way for AdamW and Muon. The one edge-of-stability event that lines up with a change in function, Muon's auxiliary AdamW overshooting its threshold just before Muon memorizes on Shakespeare, is not what drives the memorization, which persists in the matched control (where the overshoot comes later and is smaller). The anatomy adds that the sharp direction is optimizer-specific and mostly made of curvature between blocks, but its changes do not line up with changes in function distance either.

Limitations.

- The noise floor varies only the data order; a second initialization (Section 5.9) gives the same distance as a new data order, but only two of each were run, on Shakespeare and TinyStories-10M.
- Learning rates come from five-point sweeps on one data order, and the Shakespeare AdamW rate is the smallest of its grid. The other rates of the sweeps agree with the tuned ones on TinyStories (Fig. 14), but only against the tuned runs' floor. SGD trains poorly at the stable rates on Shakespeare, so comparisons with SGD there stop at loss 2.61, and its large data-order noise on TinyStories makes "no split" against SGD a weak statement.
- The first exceedance is degenerate (Section 4.2); episodes and the persistent split depend on the margin $m_\times$, the run length k and the matching tolerance τ; the conclusions hold over the variants we tried (Section 5.2), except the Shakespeare Muon–SGD split.
- The loss grid ends at the lowest loss that all runs of a pair reach, so a split below it would be missed.
- The thresholds are those of linear stability on a quadratic with a frozen preconditioner; we have no threshold for Muon, and the AdamW ratio at the first steps reflects a preconditioner built from few gradients.
- Sharpness is measured on a fixed batch of 32 sequences with 20 Lanczos iterations, in float32, while training uses bfloat16 autocast.
- Loss matching and the mode-connectivity barrier use a training probe of 64 sequences.
- The landing coordinate is a ratio of distances, not a projection onto the segment between the pure runs.
- The models are small (up to 30M parameters) and trained for at most 655M tokens.

7 Conclusion

From the same initialization and data order, SGD with momentum, AdamW and Muon do not first share a path and then split. At equal training loss they differ most at the start, before data-order noise has built up, and from about

the bigram stage on their predictions are as close as those of one optimizer trained on two data orders. This holds in all three language-model settings and for every pair except those with SGD on Shakespeare, where SGD barely trains, and apart from one short AdamW–Muon episode on Shakespeare. At the end of TinyStories-30M, replacing AdamW by Muon changes the function no more than reshuffling the data. Neither the edge of stability nor the end of warmup explains these transitions. Switching optimizers during training does not reveal an early decision either: the run ends at the function of the optimizer it switches to whenever that one is fast enough to get there, on TinyStories whenever it is the faster one, even late in training. The optimizer does leave a mark in weight space: after a change of optimizer a fork usually needs a later fork step to stay linearly connected to its parent, the barrier between AdamW and Muon extends beyond the first block, where that between two data orders lies, and the weights carry each optimizer's geometry, mixed after a swap according to how far each moved them. On TinyStories the hidden representations differ after the first block and little after the others, and the functions no more than with another data order even off the data distribution. As a check, on TinyStories every rate of the sweeps that trains stably is within 0.8–1.1 floors of the tuned runs, with one exception at 1.35. A linear model on separable data, where the limits of training are known to differ, shows that our test can detect a split, and places it near interpolation; but AdamW and Muon trained on Shakespeare until they memorize the training set do not split either, so for these models the absence of a split is not only a matter of training too briefly.

References

Adilova, L., M. Andriushchenko, M. Kamp, et al. (2024). "Layer-wise Linear Mode Connectivity". In: *ICLR*.

Andreyev, A., A. Ananthkumar, M. Walden, et al. (2026). *Momentum Further Constrains Sharpness at the Edge of Stochastic Stability*. Preprint.

Andreyev, A. and P. Beneventano (2024). *Edge of Stochastic Stability: Revisiting the Edge of Stability for SGD*. Preprint.

Bartlett, P. L., D. J. Foster, and M. Telgarsky (2017). "Spectrally-normalized Margin Bounds for Neural Networks". In: *NeurIPS*.

Cattaneo, M. D., J. M. Klusowski, and B. Shigida (2024). "On the Implicit Bias of Adam". In: *ICML*.

Cohen, J. M., A. Damian, A. Talwalkar, et al. (2025). "Understanding Optimization in Deep Learning with Central Flows". In: *ICLR*.

Cohen, J. M., B. Ghorbani, S. Krishnan, et al. (2022). *Adaptive Gradient Methods at the Edge of Stability*. Preprint.

Cohen, J. M., S. Kaur, Y. Li, et al. (2021). "Gradient Descent on Neural Networks Typically Occurs at the Edge of Stability". In: *ICLR*.

Cortesi, F. V., G. Iannone, G. Crippa, et al. (2026). *Same Error, Different Function: The Optimizer as an Implicit Prior in Financial Time Series*. Preprint.

Damian, A., E. Nichani, and J. D. Lee (2023). "Self-Stabilization: The Implicit Bias of Gradient Descent at the Edge of Stability". In: *ICLR*.

Eldan, R. and Y. Li (2023). *TinyStories: How Small Can Language Models Be and Still Speak Coherent English?*

Fan, C., M. Schmidt, and C. Thrampoulidis (2025). "Implicit Bias of Spectral Descent and Muon on Multiclass Separable Data". In: *NeurIPS*.

Ford, N., J. Gilmer, N. Carlini, and E. D. Cubuk (2019). "Adversarial Examples Are a Natural Consequence of Test Error in Noise". In: *ICML*.

Frankle, J., G. K. Dziugaite, D. M. Roy, and M. Carbin (2020). "Linear Mode Connectivity and the Lottery Ticket Hypothesis". In: *ICML*.

Gunasekar, S., J. Lee, D. Soudry, and N. Srebro (2018). "Characterizing Implicit Bias in Terms of Optimization Geometry". In: *ICML*.

Hahn, M., D. Jurafsky, and R. Futrell (2021). "Sensitivity as a Complexity Measure for Sequence Classification Tasks". In: *Transactions of the Association for Computational Linguistics*.

Jastrzebski, S., M. Szymczak, S. Fort, et al. (2020). "The Break-Even Point on Optimization Trajectories of Deep Neural Networks". In: *ICLR*.

Jiang, Y., B. Neyshabur, H. Mobahi, et al. (2020). "Fantastic Generalization Measures and Where to Find Them". In: *ICLR*.

Jordan, K., Y. Jin, V. Boza, et al. (2024). *Muon: An Optimizer for Hidden Layers in Neural Networks*. Blog post. URL: https://kellerjordan.github.io/posts/muon/.

Kalra, D. S. and M. Barkeshli (2024). "Why Warmup the Learning Rate? Underlying Mechanisms and Improvements". In: *NeurIPS*.

Karpathy, A. (2022). *nanoGPT*. Software. URL: https://github.com/karpathy/nanoGPT.

Keskar, N. S. and R. Socher (2017). *Improving Generalization Performance by Switching from Adam to SGD*. Preprint.

Kornblith, S., M. Norouzi, H. Lee, and G. Hinton (2019). "Similarity of Neural Network Representations Revisited". In: *ICML*.

Krizhevsky, A. (2009). *Learning Multiple Layers of Features from Tiny Images*. Tech. rep. University of Toronto.

Kunstner, F., A. Milligan, R. Yadav, et al. (2024). "Heavy-Tailed Class Imbalance and Why Adam Outperforms Gradient Descent on Language Models". In: *Advances in Neural Information Processing Systems*.

Liu, J., J. Su, X. Yao, et al. (2025). *Muon is Scalable for LLM Training*. Preprint.

Moosavi-Dezfooli, S.-M., A. Fawzi, J. Uesato, and P. Frossard (2019). "Robustness via Curvature Regularization, and Vice Versa". In: *CVPR*.

Nagarajan, V. and J. Z. Kolter (2019). *Generalization in Deep Networks: The Role of Distance from Initialization*. NeurIPS 2017 Workshop on Deep Learning: Bridging Theory and Practice.

Neyshabur, B., R. Tomioka, and N. Srebro (2015). "Norm-Based Capacity Control in Neural Networks". In: *COLT*.

Novak, R., Y. Bahri, D. A. Abolafia, et al. (2018). "Sensitivity and Generalization in Neural Networks: an Empirical Study". In: *ICLR*.

Sanyal, A., P. H. S. Torr, and P. K. Dokania (2020). "Stable Rank Normalization for Improved Generalization in Neural Networks and GANs". In: *ICLR*.

Smith, S. L., B. Dherin, D. G. T. Barrett, and S. De (2021). "On the Origin of Implicit Regularization in Stochastic Gradient Descent". In: *ICLR*.

Soudry, D., E. Hoffer, M. S. Nacson, et al. (2018). "The Implicit Bias of Gradient Descent on Separable Data". In: *JMLR*.

Srinivas, S., K. Matoba, H. Lakkaraju, and F. Fleuret (2022). "Efficient Training of Low-Curvature Neural Networks". In: *NeurIPS*.

Vasudeva, B., J. W. Lee, V. Sharan, and M. Soltanolkotabi (2025). "The Rich and the Simple: On the Implicit Bias of Adam and SGD". In: *NeurIPS*.

Wilson, A. C., R. Roelofs, M. Stern, et al. (2017). "The Marginal Value of Adaptive Gradient Methods in Machine Learning". In: *NeurIPS*.

Xie, S. and Z. Li (2024). "Implicit Bias of AdamW: ℓ_∞ Norm Constrained Optimization". In: *ICML*.

Zhang, C., D. Zou, and Y. Cao (2024a). "The Implicit Bias of Adam on Separable Data". In: *NeurIPS*.

Zhang, Y., C. Chen, T. Ding, et al. (2024b). "Why Transformers Need Adam: A Hessian Perspective". In: *NeurIPS*.

Zhao, R., D. Morwani, D. Brandfonbrener, et al. (2025). "Deconstructing What Makes a Good Optimizer for Language Models". In: *ICLR*.

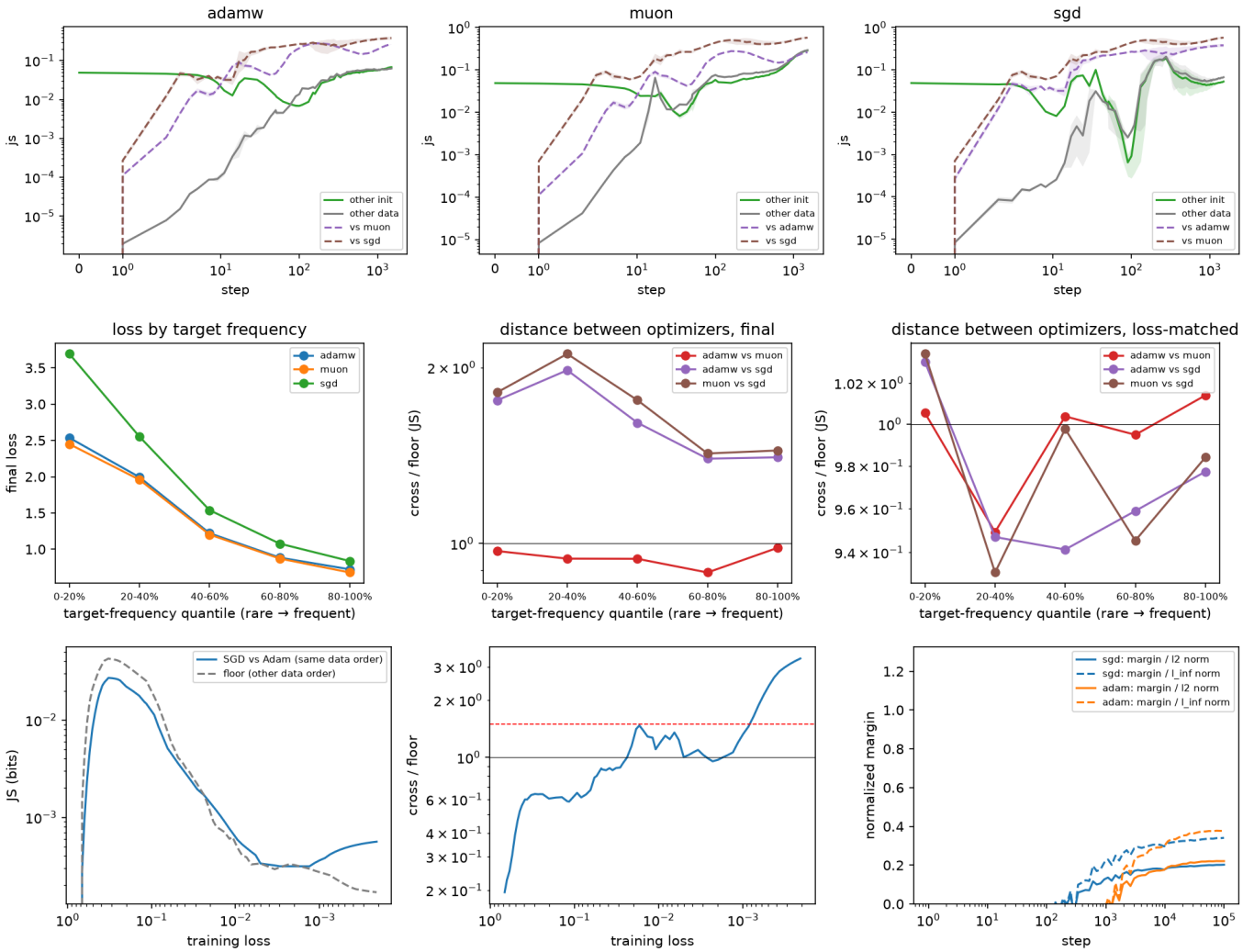


Figure 13: Controls added after the plan. Top: second initialization on Shakespeare; for each optimizer, at equal steps, the distance to the same optimizer from the other initialization (same data order), to the same optimizer with the other data order (same initialization), and to the other optimizers (same initialization and data order); median and range over the pairs. Middle: TinyStories-10M by target-token frequency (quintiles of the training frequency of the target, rare to frequent): final validation loss of each optimizer (mean over data orders), and the distance between optimizers over the floor at the final step and at the lowest training loss both runs reach. Bottom: the linear positive control; loss-matched JS distance between SGD and Adam on the test points against the data-order floor (left), their ratio with the margin $1 + m_\times = 1.5$ (middle), and the ℓ_2- and ℓ_∞-normalized margins over training (right).

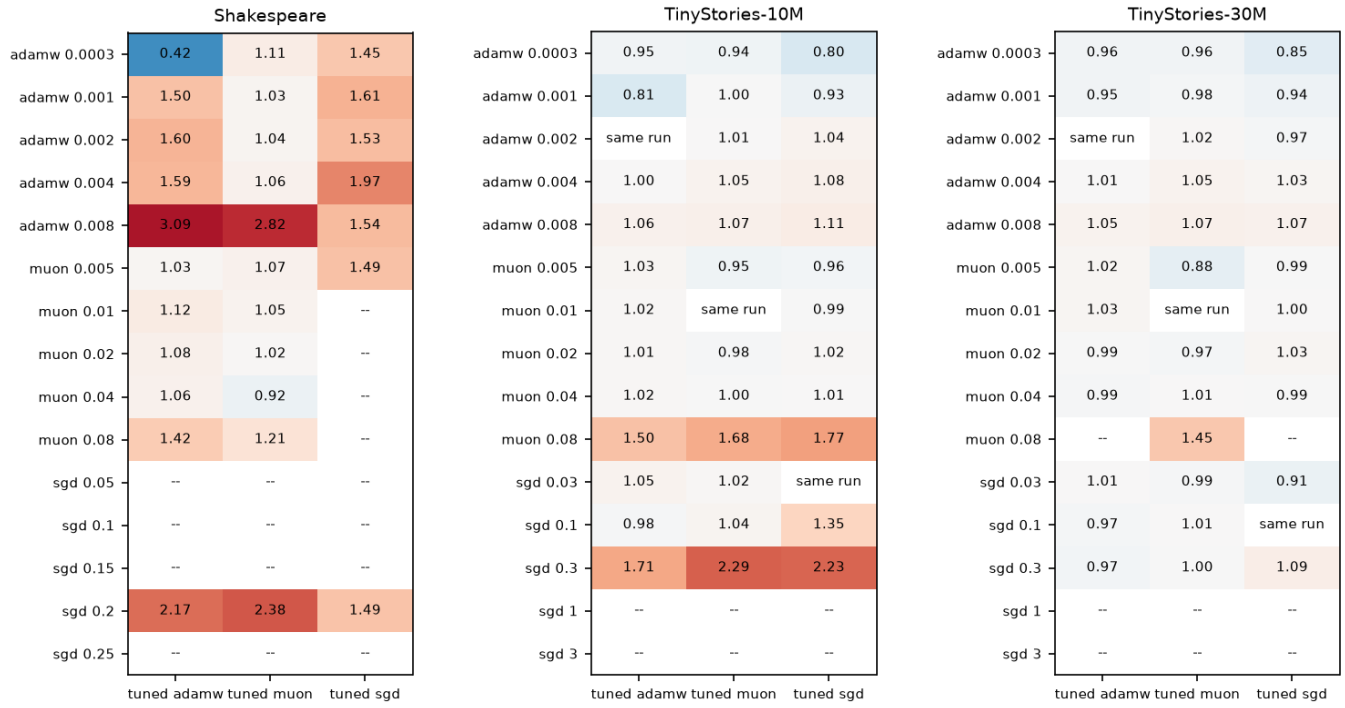


Figure 14: Each learning-rate sweep run (rows) against the tuned run of each optimizer (columns) on the same data order: median loss-matched JS over the data-order floor of the tuned runs, below the bigram loss (2.47 on Shakespeare, 3.8 on TinyStories). Blue below 1, red above; "–": diverged or never below that loss.

A Hyperparameters

Table 4: Optimizer hyperparameters, shared by all language-model settings unless noted. The Shakespeare control matches Muon's auxiliary AdamW to AdamW: $\beta = (0.9, 0.99)$, weight decay 0.1, learning rate $3 \cdot 10^{-4}$.

	SGD	AdamW	Muon
Update	heavy ball, $\mu = 0.9$, not Nesterov	$\beta = (0.9, 0.95)$; Shakespeare $(0.9, 0.99)$; $\epsilon = 10^{-8}$	Nesterov momentum 0.95; 5 Newton–Schulz steps (bfloat16); scale $\sqrt{\max(1, n_{\text{out}}/n_{\text{in}})}$
Parameters	all	all	matrices in transformer blocks; auxiliary AdamW ($\beta = (0.9, 0.95)$, $\epsilon = 10^{-8}$) on embeddings, head, gains
Weight decay	none	0.1 on tensors with ≥ 2 dims, none on gains	none (Muon and auxiliary AdamW)
Clipping	none	none	none

Table 5: Learning-rate sweeps (data order 0, five rates per optimizer). Rates used in the main runs in bold. Selection: final validation loss for TinyStories, minimum validation loss over training for Shakespeare. The auxiliary AdamW of Muon keeps the learning rate of Table 1 throughout the sweep. SGD (†: best on data order 0) uses a lower rate, since its best rate diverged or blew up on other data orders: 0.2 on Shakespeare (orders 1 and 2), 0.1 on TinyStories-10M (orders 1–3) and 0.3 on TinyStories-30M (orders 2–4).

Setting	SGD	AdamW	Muon
Shakespeare-char	**0.05**, 0.1, 0.15, 0.2†, 0.25	$\mathbf{3 \cdot 10^{-4}}$, 10^{-3}, $2 \cdot 10^{-3}$, $4 \cdot 10^{-3}$, $8 \cdot 10^{-3}$	0.005, 0.01, 0.02, **0.04**, 0.08
TinyStories-10M	**0.03**, 0.1†, 0.3, 1, 3	$3 \cdot 10^{-4}$, 10^{-3}, $\mathbf{2 \cdot 10^{-3}}$, $4 \cdot 10^{-3}$, $8 \cdot 10^{-3}$	0.005, **0.01**, 0.02, 0.04, 0.08
TinyStories-30M	0.03, **0.1**, 0.3†, 1, 3	$3 \cdot 10^{-4}$, 10^{-3}, $\mathbf{2 \cdot 10^{-3}}$, $4 \cdot 10^{-3}$, $8 \cdot 10^{-3}$	0.005, **0.01**, 0.02, 0.04, 0.08

Table 6: Fork steps of the swaps (all six ordered pairs, plus each optimizer into itself with data order 1).

Setting	Fork steps
Shakespeare-char	25, 50, 100, 200, 300, 450, 600, 800, 1000
TinyStories-10M	50, 100, 200, 400, 700, 1000, 1500, 2500, 4000
TinyStories-30M	100, 200, 400, 800, 1400, 2000, 3000, 5000, 8000

Break-even runs: plain SGD (momentum 0), constant learning rate, no warmup. CIFAR-10: 100 epochs, i.e. 39,000 steps at batch 128, 166,600 at batch 30 and 16,600 at batch 300; data orders 0 and 1. Shakespeare and TinyStories-10M: 1,500 and 10,000 steps at their training batch size, learning rates 0.03, 0.1 and 0.3; data orders 0, 1 and 2.

B Measurement details

A grid "every $N + K$ log" is $\{0, N, 2N, \dots\} \cup \{\text{round}(T^u) : u \in K \text{ evenly spaced values in } [0, 1]\} \cup \{T\}$, i.e. the union of a linear grid and K log-spaced steps between 1 and T.

Sharpness. Lanczos runs for 20 iterations with full reorthogonalization (two Gram–Schmidt passes per iteration) from a fixed Gaussian start, and returns the top Ritz pair. Hessian-vector products use double backpropagation through one gradient graph, with the exact (math) attention kernel, in float32 without autocast, on the model before the update of the measured step.

Split. $\tau = 0.02$, $m_\times = 0.5$, $m_+ = 0$, $k = 3$, 1000 bootstrap resamples. The training loss of a run is its loss on the training probe at its prediction steps.

Mode connectivity. Barrier on the 64 sequences of the training probe, at 11 evenly spaced values of α, between final checkpoints.

Prediction trajectories. 128 evenly spaced probe positions; top 50 principal components of the centered probability vectors, then UMAP.

Table 7: Logging grids, probe sizes and measurement settings of the main language-model runs (forks inherit their parent's) and of the CIFAR-10 runs at batch 128. Fixed sets of examples are drawn once, from random streams separate from the training batches, and are the same for all runs of a setting.

	Shakespeare-char	TinyStories-10M	TinyStories-30M	CIFAR-10 (batch 128)
Prediction steps $\mathcal{S}$	every 50 + 40 log	every 250 + 30 log	every 500 + 30 log	every 390 + 40 log
Checkpoints	every 250 + 12 log	every 1000 + 12 log	every 2000 + 12 log	every 3900 + 12 log
Measurement steps $\mathcal{M}$	every 50 + 30 log	every 500 + 30 log	every 1000 + 30 log	every 390 + 30 log
Validation probe (N)	$64 \times 256 = 16{,}384$	$128 \times 16 = 2{,}048$	$128 \times 16 = 2{,}048$	10,000 images
Training probe	64 seq. (16,384 tok.)	64 seq. (16,384 tok.)	64 seq. (16,384 tok.)	2,000 images
Hessian batch $\mathcal{H}$	32 seq. (8,192 tok.)	32 seq. (8,192 tok.)	32 seq. (8,192 tok.)	2,500 images
Lanczos iterations	20	20	20	20
Batch sharpness	8×64 seq.	8×128 seq.	8×128 seq.	8×128 images
$\hat{K}$ (break-even runs)	25×64 seq.	25×128 seq.	–	25×128 images

B.1 Function metrics

The metrics are computed on 1024 fixed validation examples (sequences or test images), drawn separately from the probe and shared by all runs; the input-derivative metrics use the first 32.

Input derivatives. Let $f(x)$ be the loss at the true labels as a function of the input x. For the language models $x \in \mathbb{R}^{64\times d}$ is the token-embedding matrix of the first 64 tokens of a sequence (position embeddings are added inside the model) and f is the mean next-token cross-entropy over these 64 positions; for the CNN x is the normalized image and f its cross-entropy. With $g = \nabla f(x)$, $H = \nabla^2 f(x)$ and $T[u,u,\cdot] = \nabla_x\big(u^\top H(x)u\big)$, and $z_1, \dots, z_8$ standard Gaussian probes:

- gradient norm $\|g\|$;
- Jacobian norm $\|J\|_F$ of the softmax output with respect to x (Novak et al., 2018): exact for the CNN (one vector-Jacobian product per class), and estimated as $\big(\frac{1}{8}\sum_j \|J^\top z_j\|^2\big)^{1/2}$ over the $64 \times V$ outputs for the language models;
- Hessian norm $\|H\|_F \approx \big(\frac{1}{8}\sum_j \|Hz_j\|^2\big)^{1/2}$;
- top input-Hessian eigenvalue $\lambda_x = v^\top Hv$, with v from 20 power iterations (the eigenvalue of largest magnitude) (Moosavi-Dezfooli et al., 2019);
- level-set curvature $\big(\frac{1}{8}\sum_j \|PHPz_j\|^2\big)^{1/2}/\|g\| \approx \|PHP\|_F/\|g\|$, with $P = I - \hat{g}\hat{g}^\top$ the projection orthogonal to the gradient, a projected variant of the normalized curvature of Srinivas et al. (2022);
- third order: the norm $\|T\|_\sigma = \max_{\|u\|=1} T[u,u,u]$, by shifted symmetric power iteration $u \leftarrow (T[u,u,\cdot] + \alpha u)/\|T[u,u,\cdot] + \alpha u\|$ with $\alpha = \|T[u_0,u_0,\cdot]\|$, 20 iterations, best of 3 random starts (as $T[u,u,u]$ is odd in u, this is also $\max|T[u,u,u]|$); the third derivative along the sharpest direction $T[v,v,v]$, with the sign of v fixed by $v^\top g \geq 0$; and the curvature length $|\lambda_x|/\|T[v,v,\cdot]\|$, the distance over which, to first order, λ_x changes by its own size (for a simple eigenvalue $\nabla_x \lambda_x = T[v,v,\cdot]$).

Each is reported as mean and standard deviation over the 32 points.

Weight norms. Over all parameters θ: ℓ_0 (entries with $|\theta_i| > 10^{-8}$), ℓ_1, ℓ_2, ℓ_3, ℓ_∞. For each weight tensor with at least two dimensions, reshaped to a matrix W (output $\times$ input): the spectral norm $\|W\|_2$, nuclear norm $\|W\|_*$ and stable rank $\|W\|_F^2/\|W\|_2^2$ (Sanyal et al., 2020). Distance from initialization $\|\theta - \theta_0\|_2$, also relative to $\|\theta_0\|_2$ (Nagarajan and Kolter, 2019). The log spectral complexity $\log\big(\prod_i \|W_i\|_2 \cdot (\sum_i (\|(W_i - W_i^0)^\top\|_{2,1}/\|W_i\|_2)^{2/3})^{3/2}\big)$ of Bartlett et al. (2017), with the initialization as reference matrices. For the CNN only, the path norm as computed by Jiang et al. (2020) (after Neyshabur et al., 2015): the sum of the outputs on an all-ones input of the network with every parameter squared and identity activations.

Accuracy and robustness. Accuracy on the 1024 validation examples (for the language models, next- token accuracy over all positions). For the CNN, accuracy under Gaussian input noise of standard deviation 0.05, 0.1 and 0.2 (in normalized pixel units) (Ford et al., 2019). For the language models, a one-token substitution sensitivity after Hahn et al. (2021): for 32 validation contexts of 64 tokens, we replace each position in turn by a uniformly random different token and count how many of the 64 substitutions change the argmax prediction at the last position; we report this count as a fraction of 64, averaged over contexts. All derivatives are taken by automatic differentiation in float32, with the exact (math) attention kernel.

C Compute

All runs used one NVIDIA L40S GPU each on the MIT ORCD cluster. The project took 103 GPU-hours in all, 60% of them for the forks (Table 8), and 89 CPU core-hours for the offline analyses that run on CPU (mode connectivity, representations, off-distribution distances, swap landing and the split statistics). Twelve smoke-test runs (0.2 GPU-hours) are not counted. The GPU-hours include runs that were later replaced and are not in the run counts (SGD at its best sweep rate on data orders 1–4, which diverged), and the Shakespeare break-even count includes the three runs at rate 1.0, which diverged.

Table 8: Runs per suite and compute. The matched-Muon control has one more run, on data order 0, that saves the fork checkpoints for the sharpness of the auxiliary block (Section 4.4).

Suite	Runs	GPU-hours
Learning-rate sweeps (3 settings)	3×15	9.0
Main runs (3 settings)	3×15	16.7
Forks (3 settings)	3×81	61.1
Break-even, Shakespeare and TinyStories-10M	$12 + 9$	2.6
Break-even, CIFAR-10	10	1.1
Shakespeare control (matched Muon)†	$5 + 1$	0.5
Second initialization, Shakespeare and TinyStories-10M†	2×6	1.0
Shakespeare trained into memorization†	10	0.5
Offline function metrics	–	10.7
Total	392	103.2

D Additional figures and tables

Table 9: Edge of stability in the language models: end of warmup w; measured steps at the edge of stability ($\mathcal{Q} \geq 0.9\,\Theta$; $\lambda_{\max}$ for SGD, preconditioned sharpness for AdamW) of the $d = 0$ run, from first to last; onset s_{EoS} over the 5 data orders; fraction of measured steps at the edge of stability over the 5 data orders; first exceedance t^*_X of the $d = 0$ run against the other two optimizers (in the order SGD, AdamW, Muon); and its persistent split, if any.

Setting	Run	w	at EoS ($d = 0$)	s_{EoS} (all d)	fraction of $\mathcal{M}$	t^*_X	$t^*_{\mathrm{p},X}$
Shakespeare	SGD	100	21–120	21	2–14%	1 / 1	191 (vs Muon)
	AdamW	100	1–500	1	11–16%	1 / 1	–
	Muon	100	–	–	–	1 / 2	14 (vs SGD)
TinyStories-10M	SGD	200	62–1500	45–500	6–12%	3 / 1	–
	AdamW	200	1–4500	1	50–65%	1 / 1	–
	Muon	200	–	–	–	1 / 7	–
TinyStories-30M	SGD	400	60–658	60–467	10–16%	2 / 1	–
	AdamW	400	1–14000	1	40–69%	1 / 1	–
	Muon	400	–	–	–	1 / 8	–

Table 10: Final function metrics on Shakespeare, mean over the five data orders, as in Table 3.

	SGD	AdamW	Muon
$\|\theta\|_2$	97	92	654
$\|\theta\|_\infty / \|\theta\|_2$ ($\times 10^3$)	12.3	12.4	2.4
stable rank (hidden, mean)	63	48	99
$\|\theta - \theta_0\| / \|\theta_0\|$	0.36	0.25	7.39
input Jacobian (Frobenius)	46.8	120.1	162.9
substitution sensitivity	0.048	0.097	0.191
top-1 accuracy	0.323	0.554	0.533

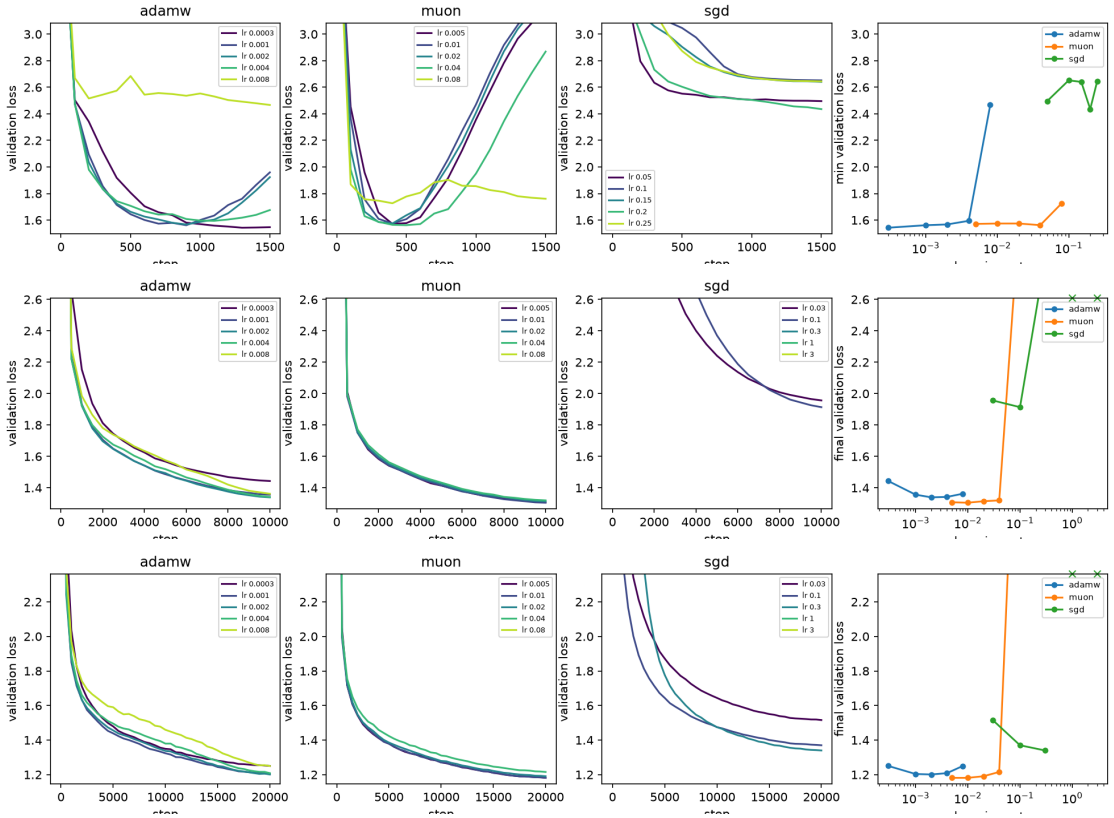


Figure 15: Learning-rate sweeps on Shakespeare (top), TinyStories-10M and TinyStories-30M (bottom), data order 0. One panel per optimizer with the validation loss of each rate against step, and a last panel with the minimum (Shakespeare) or final (TinyStories) validation loss against the learning rate; × marks runs that diverged. Curves above the plotted range are cut.

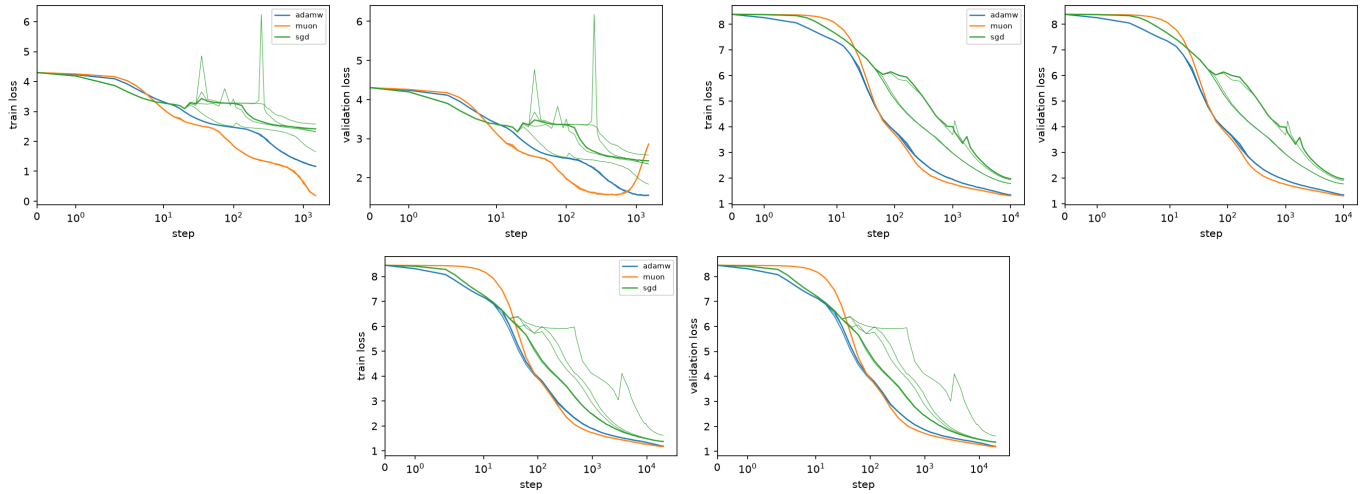


Figure 16: Training-probe and validation loss of the main runs against step: Shakespeare (top left), TinyStories-10M (top right), TinyStories-30M (bottom). Data order 0 thick, orders 1–4 thin.

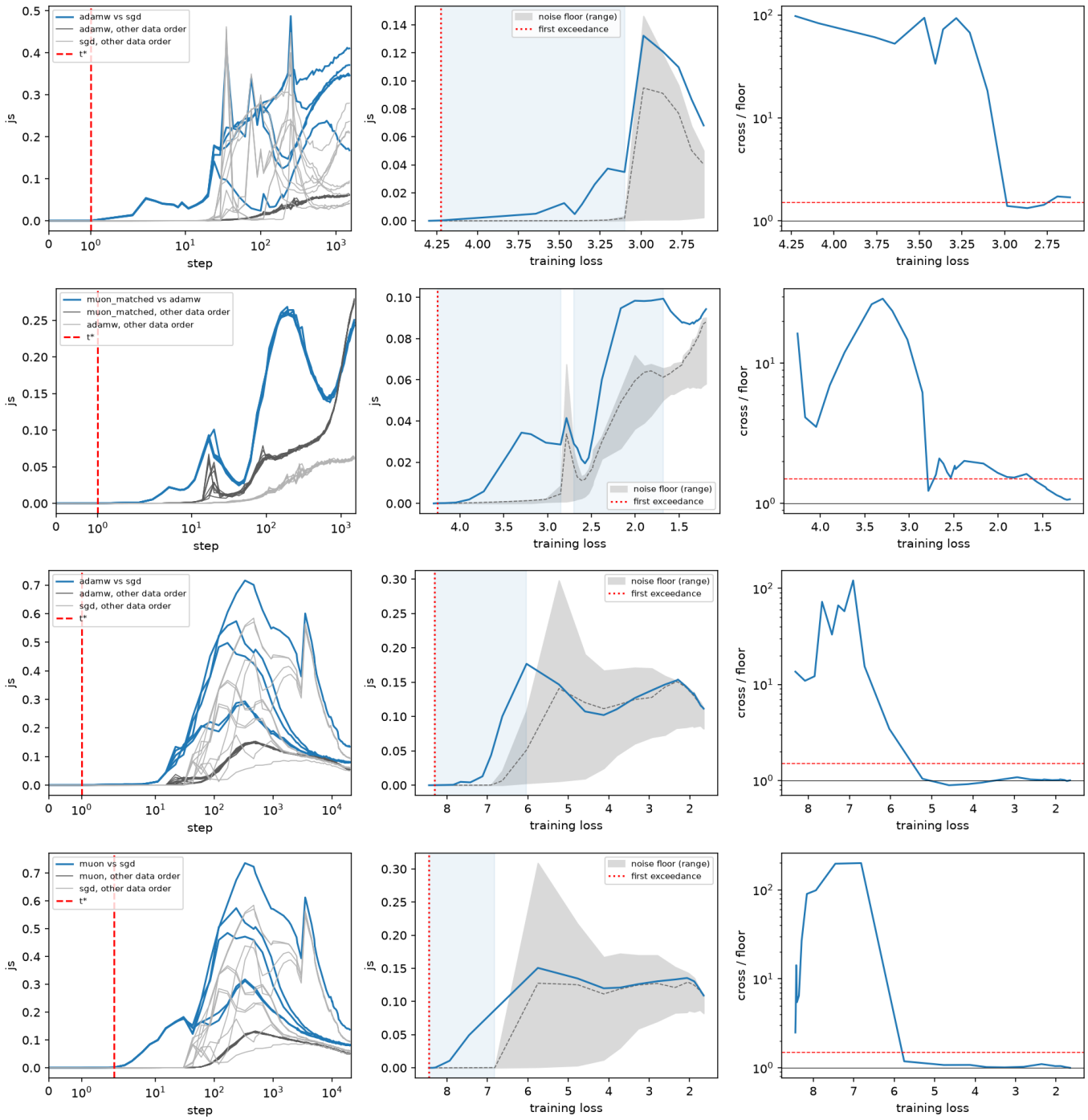


Figure 17: Divergence figures as in Fig. 2. Rows: Shakespeare AdamW–SGD; Shakespeare matched Muon–AdamW (control); TinyStories-30M AdamW–SGD; TinyStories-30M Muon–SGD.

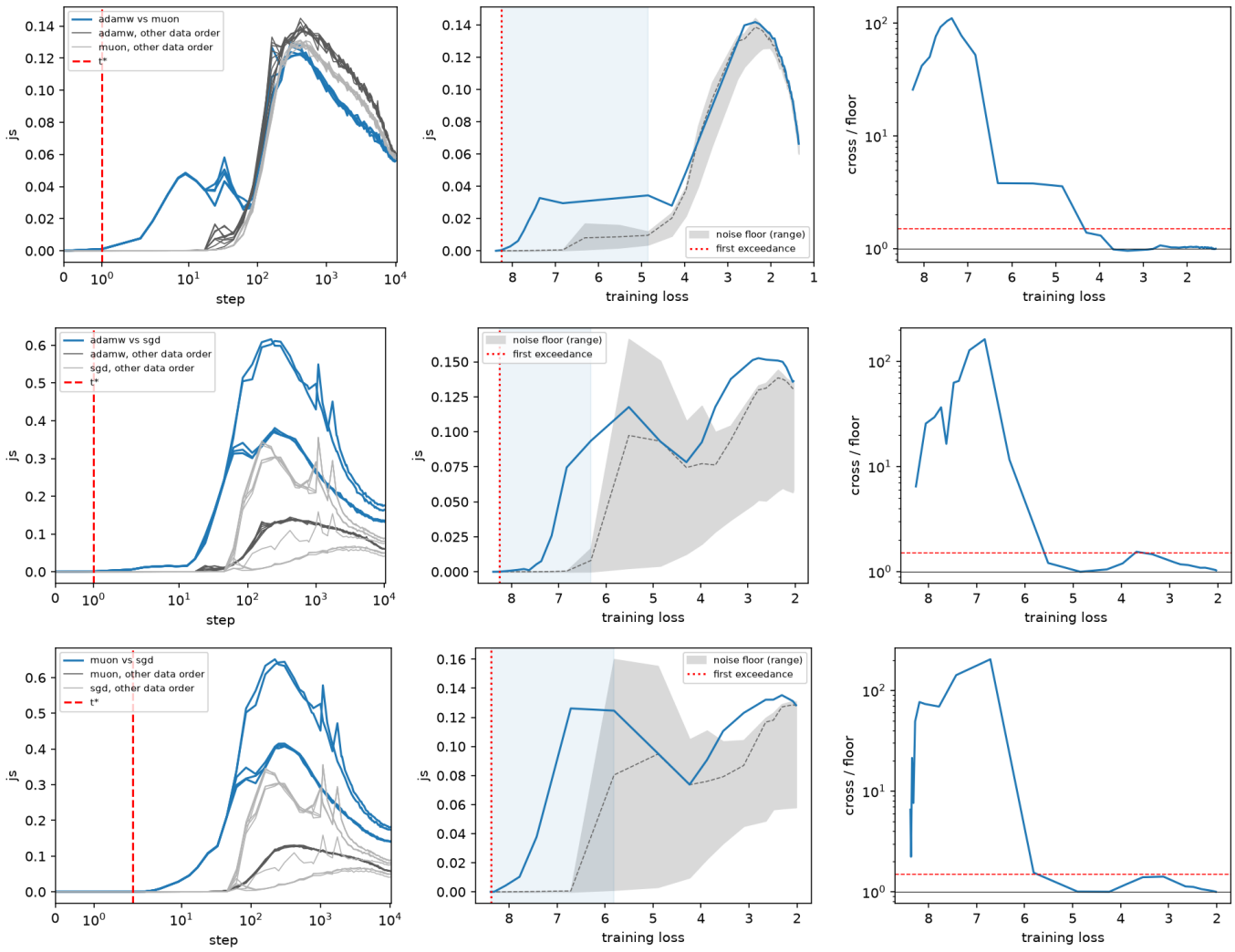


Figure 18: Divergence figures as in Fig. 2 for TinyStories-10M. Rows: AdamW–Muon, AdamW–SGD, Muon–SGD.

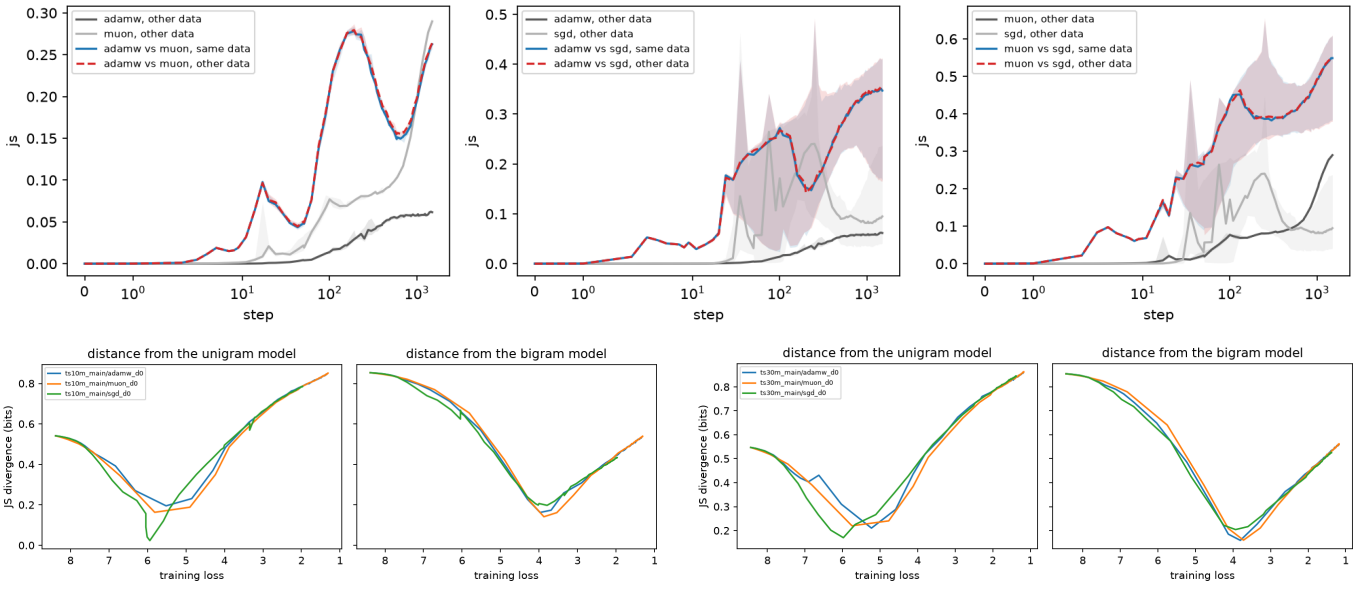


Figure 19: Top: optimizer against data order at equal steps on Shakespeare, as in Fig. 3 (top), for the three pairs. Bottom: distance of the $d = 0$ runs from the unigram and bigram models on TinyStories-10M (left) and TinyStories-30M (right), as in Fig. 3 (bottom).

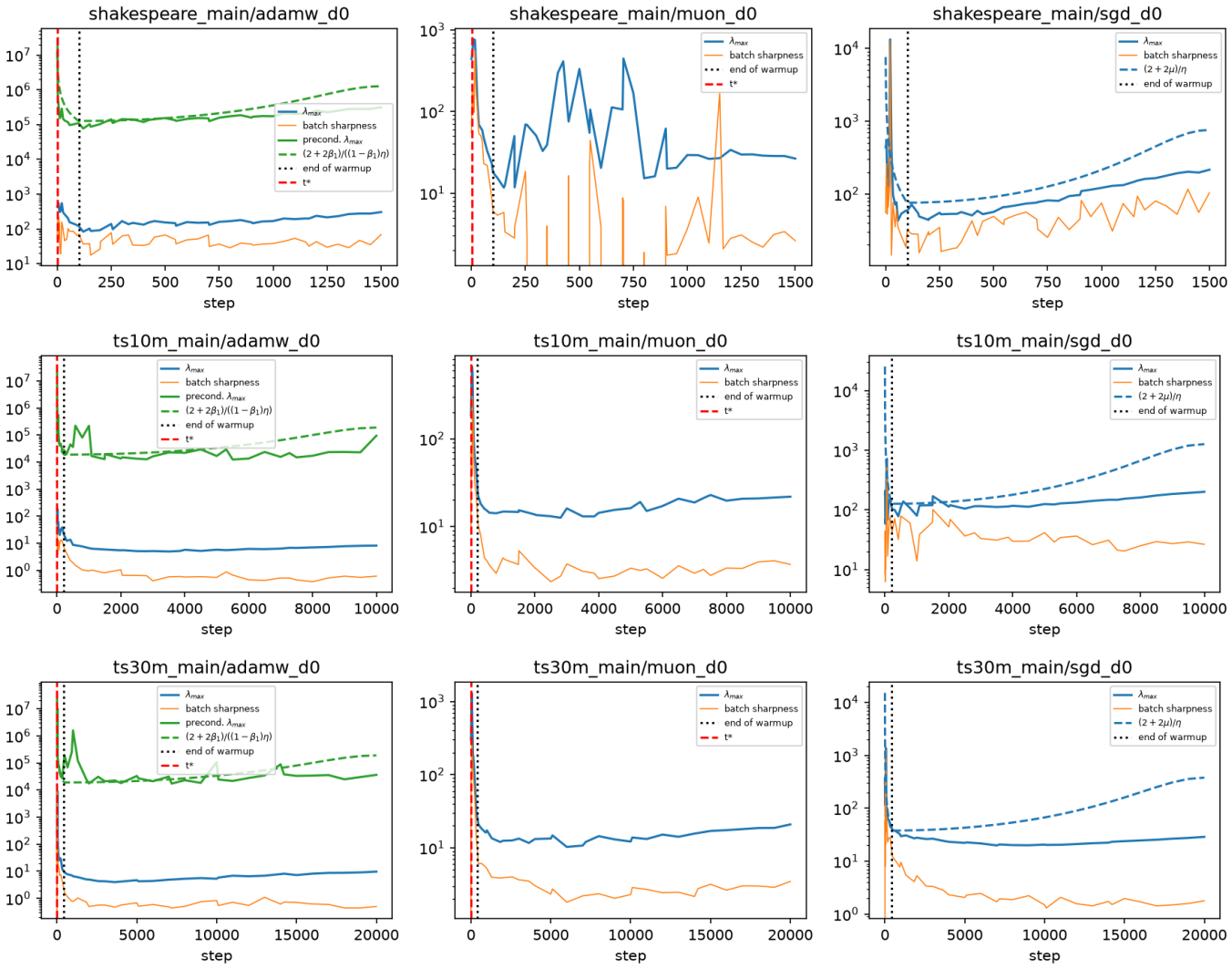


Figure 20: Sharpness of the $d = 0$ runs on Shakespeare (top), TinyStories-10M and TinyStories-30M (bottom), log scale: $\lambda_{\max}$, batch sharpness, for AdamW the preconditioned sharpness, and the thresholds $(2 + 2\mu)/\eta_s$ (SGD) and $(2 + 2\beta_1)/((1 - \beta_1)\eta_s)$ (AdamW, dashed); end of warmup dotted, first exceedance t^*_X of the AdamW–Muon pair red dashed.

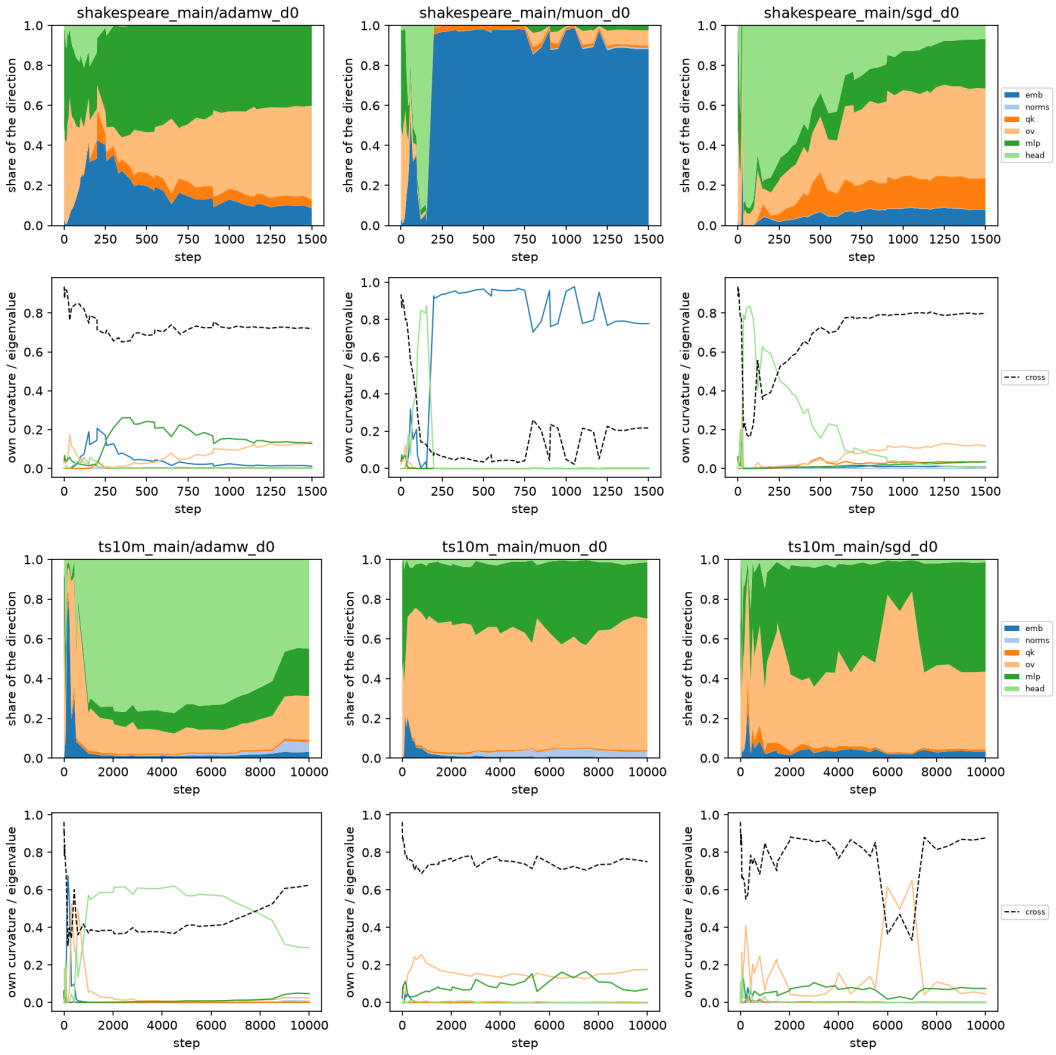


Figure 21: Anatomy of the top Hessian eigenvector as in Fig. 7, on Shakespeare (top two rows) and TinyStories-10M (bottom two rows).

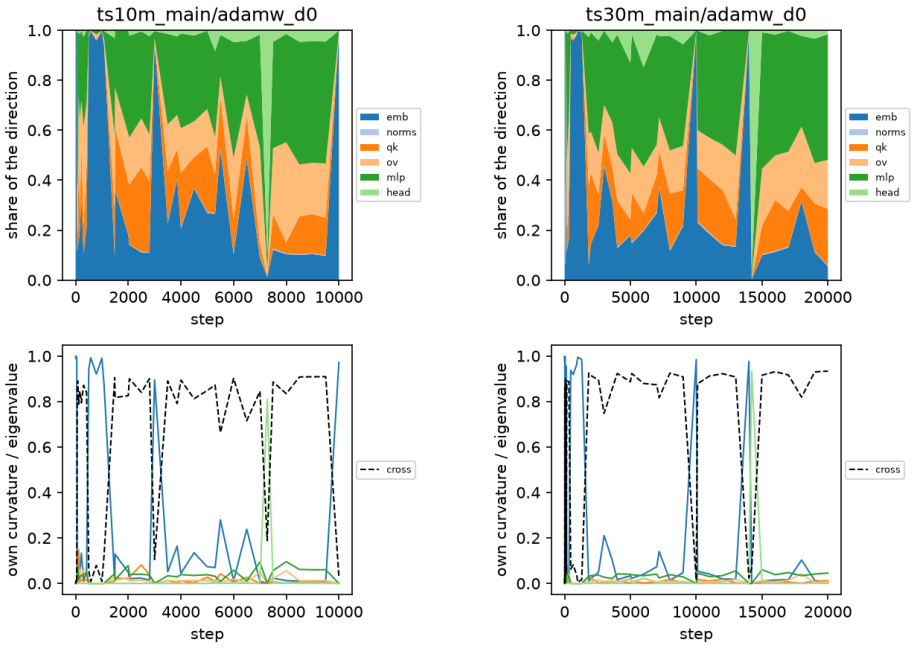


Figure 22: Anatomy of the top eigenvector of AdamW's preconditioned operator $P^{-1/2}HP^{-1/2}$ on TinyStories-10M (left) and TinyStories-30M (right), as in Fig. 7.

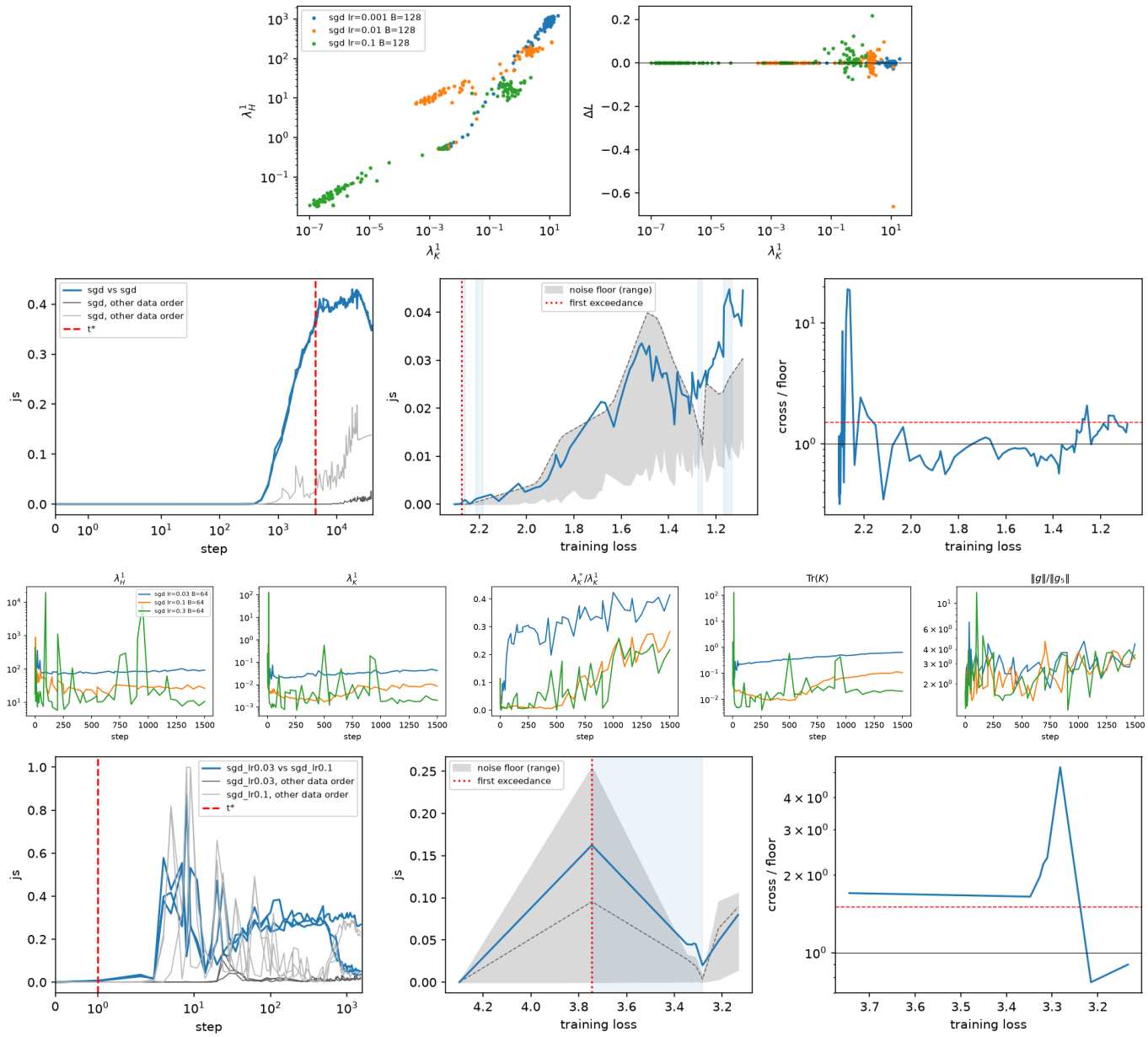


Figure 23: Break-even analysis, continued. First row: CIFAR-10, λ_H^1 (left) and ΔL (right) against λ_K^1. Second row: CIFAR-10 divergence figure for $\eta = 0.001$ against 0.01. Third and fourth rows: Shakespeare with plain SGD at $\eta \in \{0.03, 0.1, 0.3\}$, the quantities of Fig. 8 (top) and the divergence figure for 0.03 against 0.1.

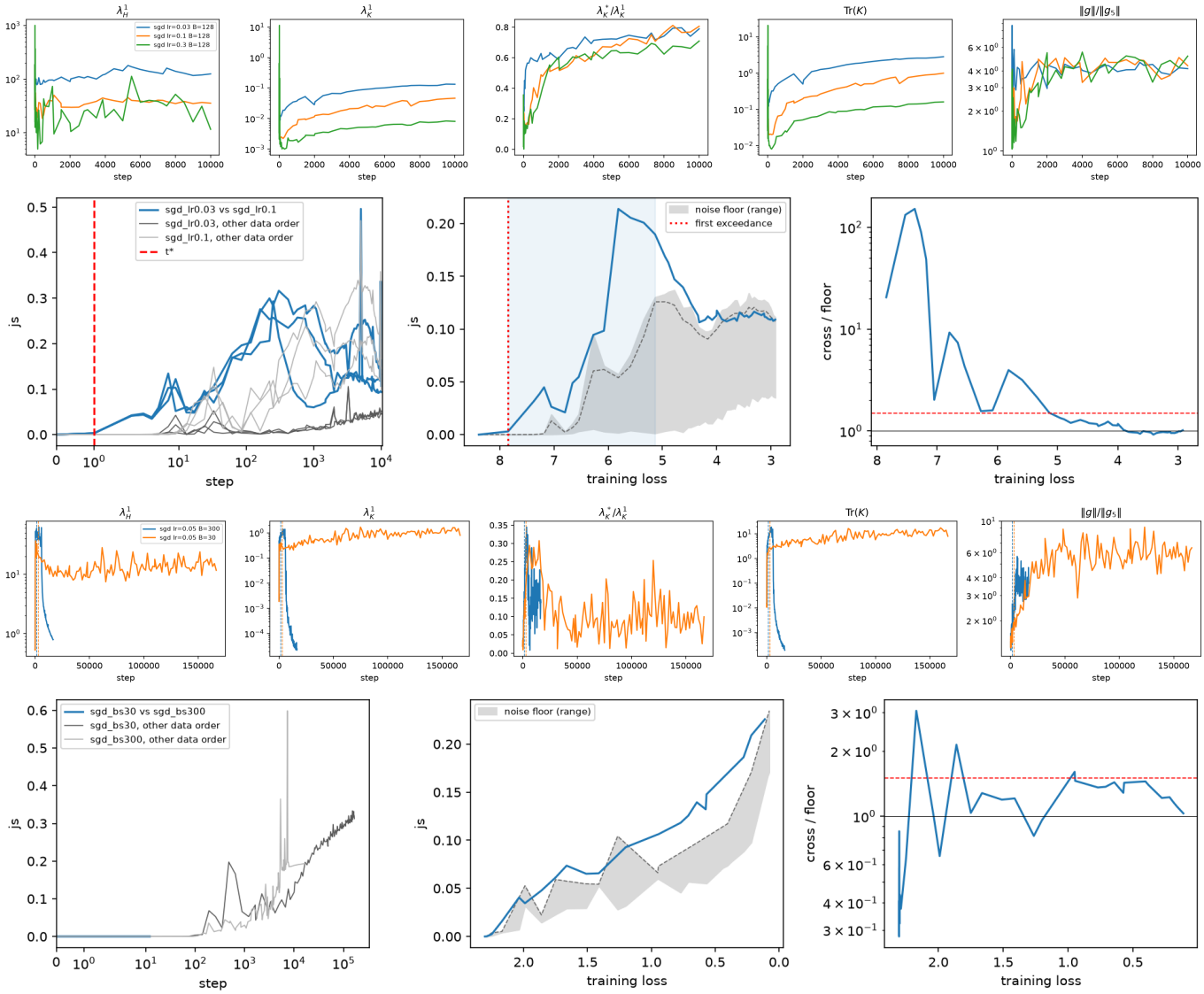


Figure 24: Break-even analysis, continued. First two rows: TinyStories-10M with plain SGD at $\eta \in \{0.03, 0.1, 0.3\}$, the quantities of Fig. 8 and the divergence figure for 0.03 against 0.1. Last two rows: CIFAR-10 at $\eta = 0.05$ with batch sizes 30 and 300.

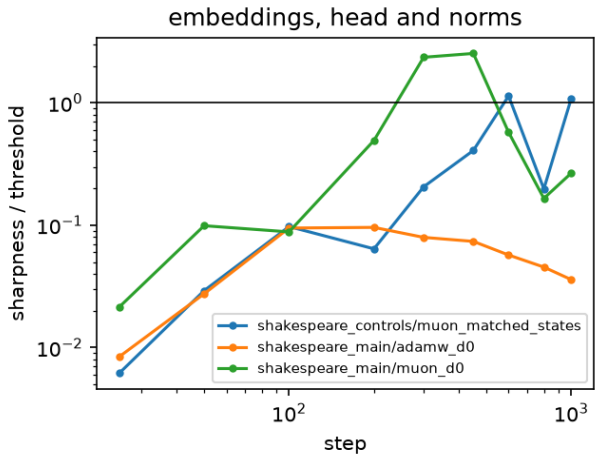


Figure 25: Sharpness of the embeddings, head and gains over their stability threshold (Section 4.4) on Shakespeare, at the fork checkpoints of AdamW, Muon, and Muon with its auxiliary AdamW matched to AdamW (learning rate $3 \cdot 10^{-4}$, betas 0.9/0.99, weight decay 0.1).

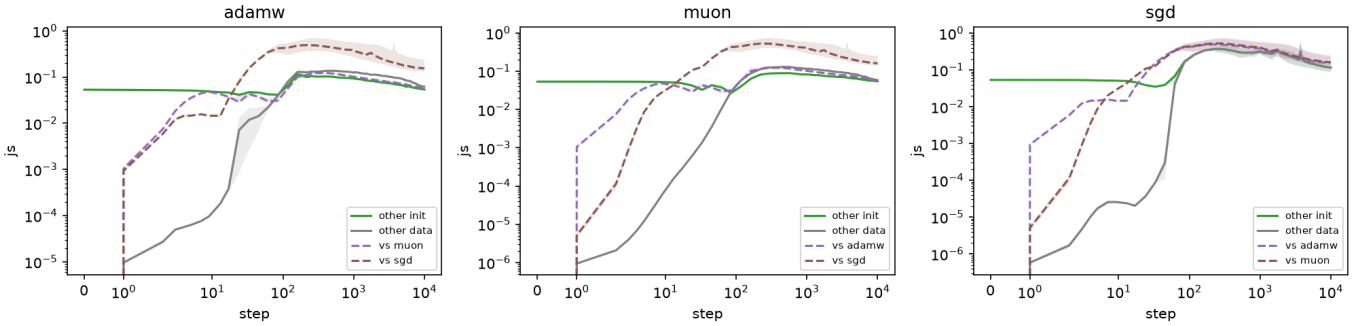


Figure 26: Second initialization on TinyStories-10M, as in Fig. 13 (top).

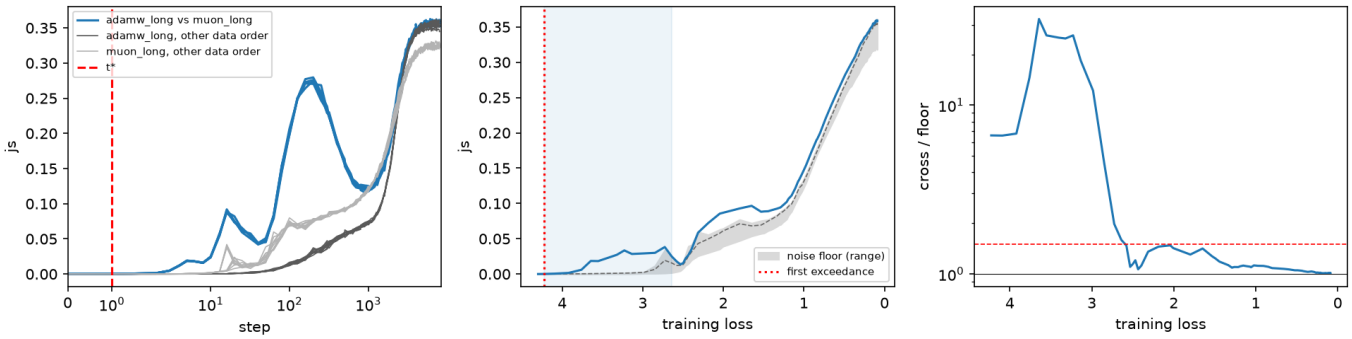


Figure 27: AdamW against Muon on Shakespeare trained for 8000 steps into memorization, as in Fig. 2.

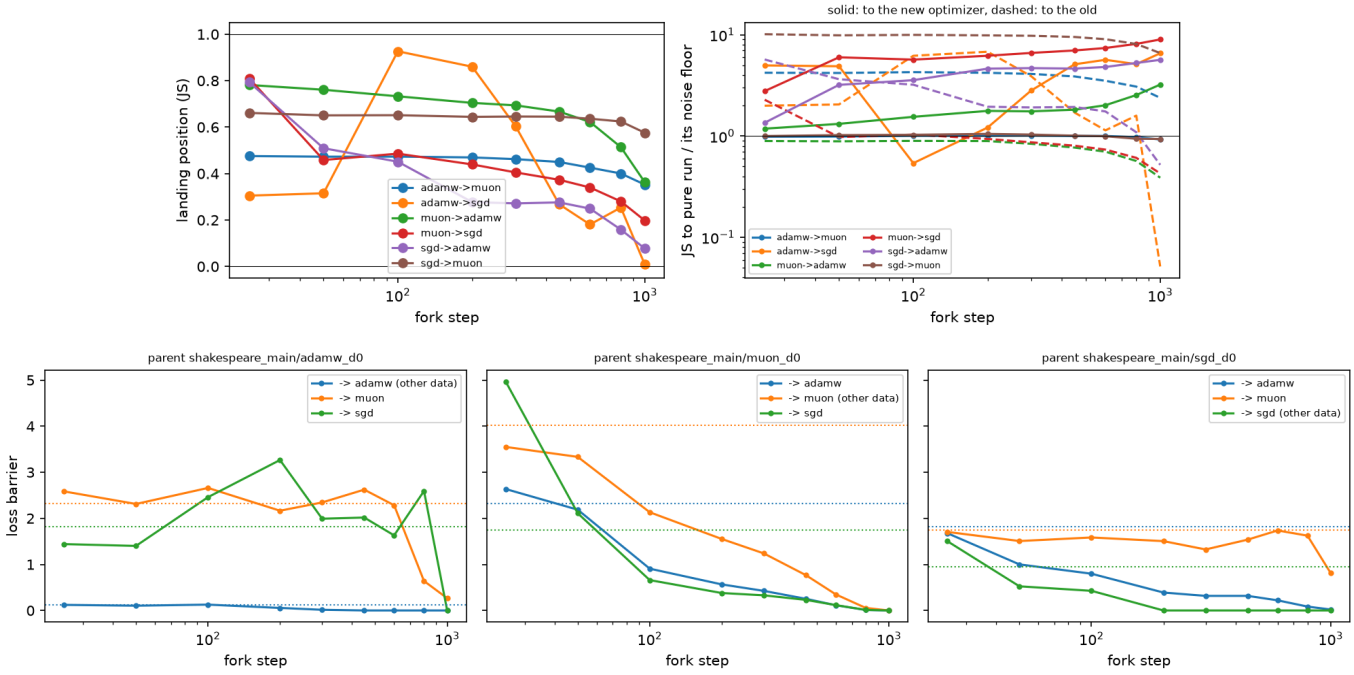


Figure 28: Swaps (top) and mode connectivity (bottom) on Shakespeare, as in Fig. 5.

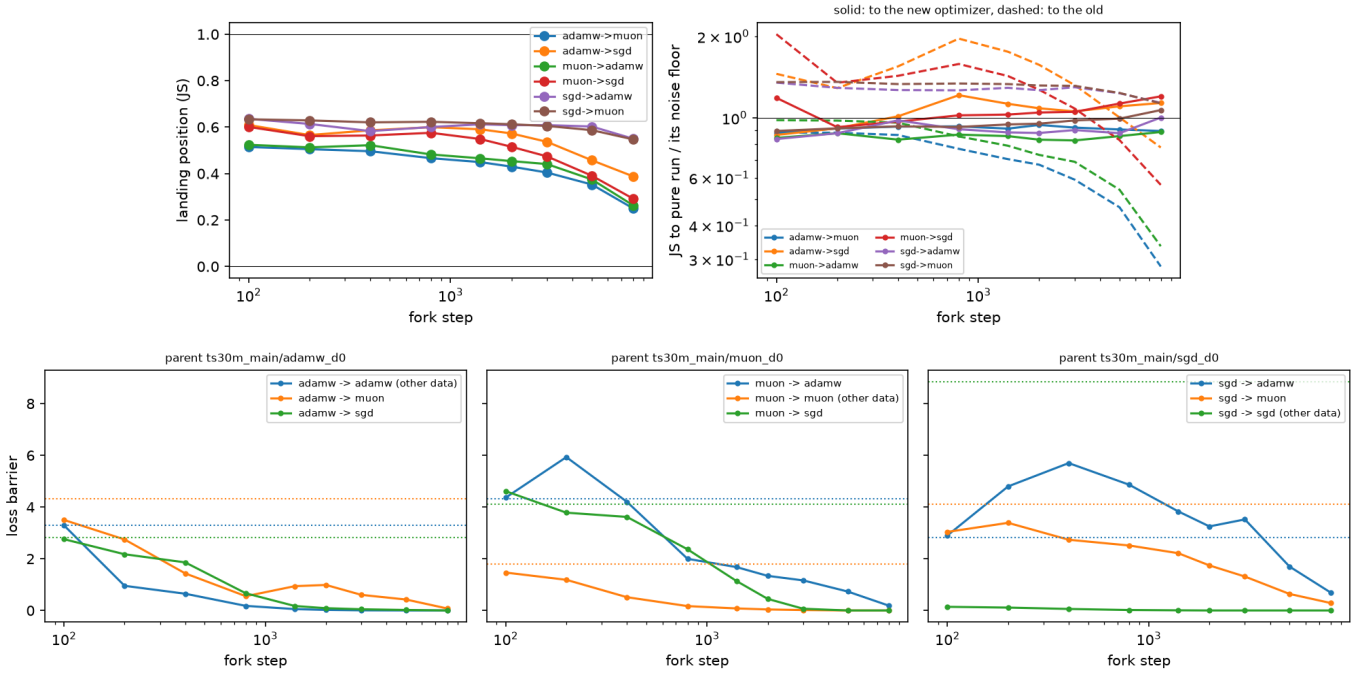


Figure 29: Swaps (top) and mode connectivity (bottom) on TinyStories-30M, as in Fig. 5.

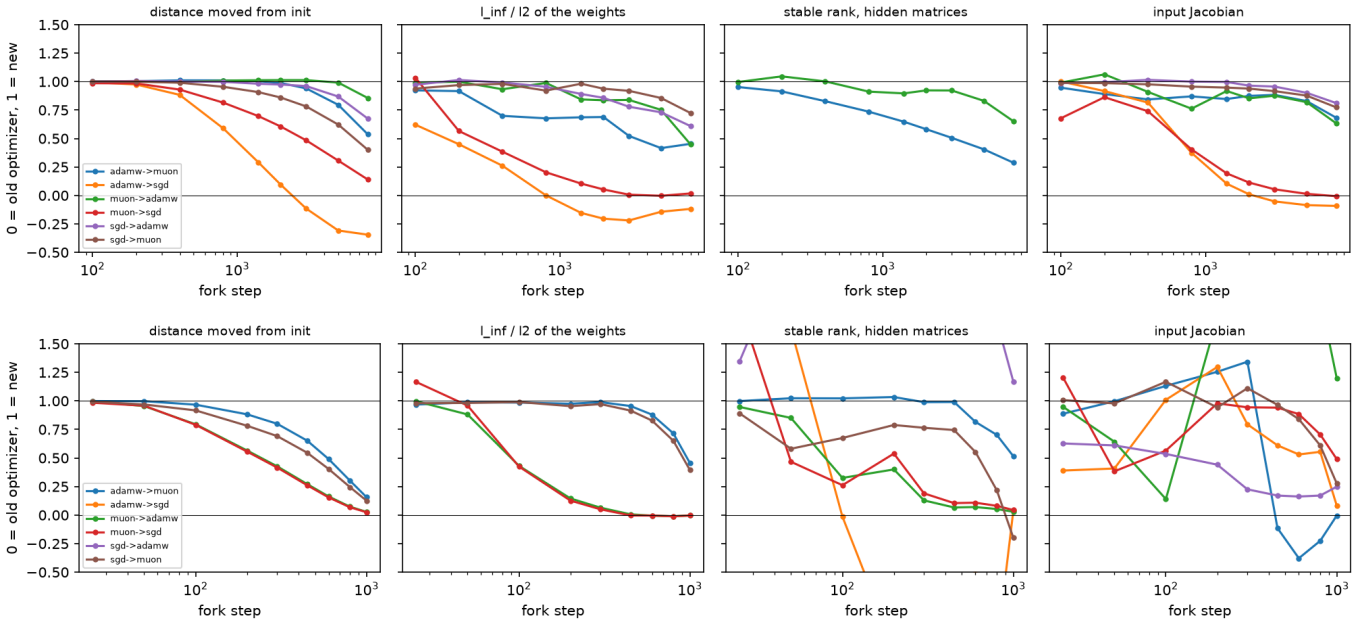


Figure 30: Final metrics of the TinyStories-30M (top) and Shakespeare (bottom) forks between pure A (0) and pure B (1), as in Fig. 12. The stable rank of SGD varies too much over data orders for the pairs with SGD to show on TinyStories-30M; on Shakespeare SGD's metrics vary widely in general, so most pairs with SGD drop out or are noisy.

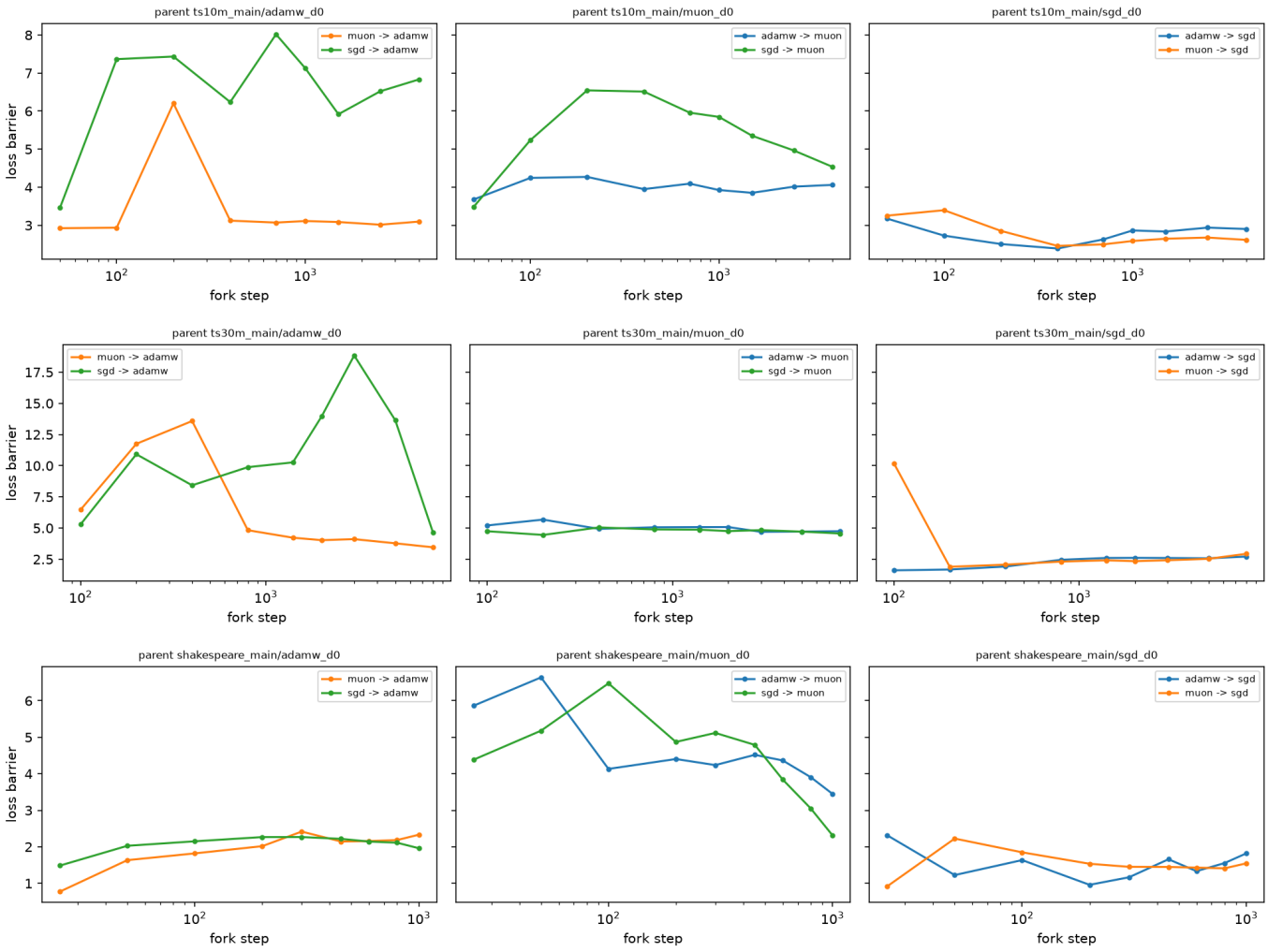


Figure 31: Barrier between the final weights of each pure $d = 0$ run and of every fork into its optimizer, against the fork step, on TinyStories-10M (top), TinyStories-30M (middle) and Shakespeare (bottom).